Federal University of Minas Gerais
School of Engineering
Bachelor's Degree in Control and Automation Engineering

MicroTEDAClus Formulation, Implementation, and Experimental Validation Reassessment and Improvement

Eduardo Henrique Basilio de Carvalho

Advisor: Prof. Janier Arias Garcia, PhD

Belo Horizonte, November 2026

Undergraduate Thesis

MicroTEDAClus Formulation, Implementation, and Experimental Validation Reassessment and Improvement

Undergraduate Thesis submitted to the examination committee designated by the Academic Board of the Bachelor's Degree in Control and Automation Engineering at the Federal University of Minas Gerais, as part of the requirements for approval in the activity Projeto Final de Curso II.

Belo Horizonte, November 2026

Resumo

MicroTEDAClus é um algoritmo de agrupamento evolutivo para fluxos de dados que resume o fluxo em microgrupos por meio da excentricidade e da tipicidade do arcabouço TEDA, e cujos autores relatam ter desempenho equivalente ao de algoritmos consolidados de agrupamento de fluxos e se adaptar melhor do que eles a mudanças nos dados. Este projeto final de curso, desenvolvido na Universidade Federal de Minas Gerais, no curso de Engenharia de Controle e Automação, sob orientação do Prof. Janier Arias Garcia, reavalia o algoritmo e propõe um modelo aprimorado. A reavaliação identifica problemas na formulação do algoritmo, diferenças entre a formulação e a implementação pública que produziu os seus resultados publicados e falhas na metodologia dos seus experimentos, entre elas etapas de pré-processamento que vazam informação sobre os dados e uma avaliação que prediz cada lote depois de aprendê-lo. Sob uma metodologia sem essas falhas, na qual todos os modelos são executados na mesma linguagem sobre os dados sem pré-processamento, avaliados de forma prequencial, ajustados pela mesma busca de hiperparâmetros e comparados por testes estatísticos em muitas execuções, o MicroTEDAClus não tem desempenho equivalente ao dos modelos de referência do trabalho original, nem se adapta melhor do que eles a mudanças de conceito.

O modelo aprimorado, MicroTEDAClus+, resulta de experimentos controlados nos quais cada correção ou modificação é aceita por uma regra definida previamente, em execuções separadas das da sua avaliação. Nos conjuntos de dados do trabalho original, ele supera o MicroTEDAClus em qualidade de agrupamento, tempo de execução e memória, nenhum modelo de referência o supera nos conjuntos com mudança de conceito e ele é o mais rápido do experimento, enquanto dois modelos de referência permanecem superiores a ele nas formas estáticas.

Palavras-chave: Agrupamento de fluxos de dados. Agrupamento evolutivo. TEDA. Mudança de conceito. Metodologia experimental.

Abstract

MicroTEDAClus is an evolving clustering algorithm for data streams that summarizes the stream into microclusters by the eccentricity and the typicality of the TEDA framework, and whose authors report that it performs on par with established stream clustering algorithms and adapts better than them to changes in the data. In this undergraduate thesis, developed at the Federal University of Minas Gerais in the Bachelor's Degree in Control and Automation Engineering, under the supervision of Prof. Janier Arias Garcia, we reassess the algorithm and propose an improved model. We find issues in the formulation of the algorithm, differences between the formulation and the public implementation that produced its published results, and flaws in the methodology of its experiments, among them preprocessing steps that leak information about the data and an evaluation that predicts each batch of samples after learning it. Under a methodology without these flaws, in which we execute every model in the same language on the unprocessed data, evaluate it prequentially, tune it by the same automatic search and compare the models over many runs by nonparametric statistical tests, MicroTEDAClus is not on par with the benchmark models of the original work, nor does it adapt better than them to concept drift.

We obtain the improved model, MicroTEDAClus+, from controlled experiments in which we accept each correction or modification by a predefined rule, on runs separate from those of its evaluation. On the datasets of the original work it improves on MicroTEDAClus in clustering quality, execution time and memory, no benchmark model is better than it in the datasets with concept drift, and it is the fastest model of the experiment, while two benchmark models remain better than it in the static shapes.

Keywords: Data stream clustering. Evolving clustering. TEDA. Concept drift. Experimental methodology.

Acknowledgments

I would like to thank Professor Janier Arias Garcia for his guidance, mentorship, and support — not only in this work but also throughout many other endeavors during my undergraduate studies. I also thank Professor Cristiano Leite de Castro for his willingness to share his experience and ideas regarding this project. Finally, I am grateful to the authors and researchers who produced and developed the knowledge and resources applied in this work.

Contents

Resumo i

Abstract iii

Acknowledgments v

List of Figures xi

List of Tables xiii

1 Introduction 1

1.1 Motivation . . . 2

1.2 Objectives . . . 2

2 Background and Related Work 3

2.1 Typicality and Eccentricity-Based Data Analytics . . . 3

2.2 Original MicroTEDAClus . . . 4

2.2.1 Issues of the Original Formulation . . . 5

2.3 Updated MicroTEDAClus . . . 6

2.4 A `C` Implementation of MicroTEDAClus . . . 7

2.4.1 Issues of the Evaluation of the C Implementation . . . 7

3 Methodology Reassessment 9

3.1 Model Implementation . . . 9

3.2 Benchmark Models . . . 11

3.3 Datasets . . . 12

3.3.1 Static Datasets . . . 12

3.3.2 Concept Drift Datasets . . . 13

3.3.3 Scalability Streams . . . 13

3.4 Evaluation Protocol . . . 14

3.5 Metrics . . . 15

3.6 Hyperparameter Selection and Runs . . . 16
3.6.1 Hyperparameter Tuning . . . 16
3.6.2 Multi-run Statistics . . . 16
3.7 Statistical Comparison . . . 17
3.8 MicroTEDAClus Under the Reassessed Methodology . . . 17
3.8.1 Clustering Quality . . . 18
3.8.2 Cost . . . 19
3.8.3 Hyperparameters . . . 19
3.9 Summary . . . 21

4 Model Improvement **22**
4.1 Experimental Design . . . 22
4.1.1 Variants and Controls . . . 22
4.1.2 Protocol and Datasets . . . 22
4.1.3 Development and Validation Runs . . . 23
4.1.4 Comparisons and Decision Rule . . . 23
4.2 Implementation Optimization . . . 24
4.2.1 Clustering Quality . . . 24
4.2.2 Execution Time . . . 25
4.3 Corrections . . . 26
4.3.1 Corrections One at a Time . . . 27
4.3.2 Combined Corrections . . . 28
4.3.3 The Corrected Model . . . 29
4.4 Improvements . . . 30
4.4.1 Improvements One at a Time . . . 32
4.4.2 Combined Improvements . . . 34
4.4.3 Refinements of the Improved Model . . . 36
4.4.4 Variants Not Retained . . . 38
4.5 Validation . . . 39
4.5.1 Clustering Quality . . . 39
4.5.2 Cost . . . 40
4.6 Limits of the Hyperparameter Selection . . . 42
4.7 MicroTEDAClus+ . . . 43
4.7.1 Microclusters . . . 43
4.7.2 Macroclusters . . . 44
4.7.3 Prediction . . . 45
4.7.4 What Remains of the Original Model . . . 45

4.8 MicroTEDAClus+ in the Main Experiment 46
4.8.1 Clustering Quality 46
4.8.2 Cost 48
4.8.3 Hyperparameters of MicroTEDAClus+ 50
4.8.4 Summary 50

5 Conclusions **52**
5.1 Final Remarks 52
5.2 Limits of the Evidence 53
5.3 Future Work 53

References **54**

List of Figures

3.1 Execution time and memory of the benchmark models of the original work through each interface 11
3.2 DS3 raw and as the original work used it, with the ARI and the times of MicroTEDAClus 13

4.1 ARI of each evaluation batch for the accepted models 40
4.2 ARI of MicroTEDAClus+, MicroTEDAClus and the benchmark models in the scalability streams 48

List of Tables

3.1 The datasets of the main experiment . . . 12
3.2 Clustering quality of MicroTEDAClus and the benchmark models . . 18
3.3 Clustering quality of MicroTEDAClus and the benchmark models in the scalability streams . . . 19
3.4 Execution time and memory footprint of MicroTEDAClus and the benchmark models . . . 20
3.5 Hyperparameters selected for MicroTEDAClus . . . 20

4.1 Implementation optimization: clustering quality . . . 25
4.2 Hyperparameter search of the two implementations . . . 25
4.3 Execution time of the two implementations with the same hyperparameters . . . 26
4.4 Candidate corrections . . . 27
4.5 Corrections, one at a time: clustering quality . . . 28
4.6 Combined corrections: clustering quality . . . 29
4.7 Combined corrections: predicted partitions . . . 29
4.8 Partitions predicted by the *corrected* model . . . 30
4.9 Search range of the variance limit: clustering quality . . . 31
4.10 Search range of the variance limit: selected limits . . . 31
4.11 Candidate improvements . . . 32
4.12 Improvements, one at a time: clustering quality . . . 33
4.13 Hyperparameters and noise of *noise-label* . . . 33
4.14 Improvements, one at a time: predictions of a single cluster . . . 34
4.15 Combined improvements: clustering quality . . . 35
4.16 Direct comparisons between combinations: clustering quality . . . 36
4.17 Refinements of the improved model: clustering quality . . . 37
4.18 Two hyperparameter searches of a single algorithm . . . 37
4.19 Variants not retained: clustering quality . . . 38
4.20 Clustering quality of the accepted models . . . 39

4.21 MicroTEDAClus+ against the *vectorized* model 40
4.22 Cost of the original implementation and of the accepted models 41
4.23 Reachability probe of two search spaces 43
4.24 Differences between MicroTEDAClus+ and MicroTEDAClus 44
4.25 Clustering quality of MicroTEDAClus+, MicroTEDAClus and the benchmark models 46
4.26 Clustering quality of MicroTEDAClus+, MicroTEDAClus and the benchmark models in the scalability streams 47
4.27 Execution time and memory footprint of MicroTEDAClus+, MicroTEDAClus and the benchmark models 49
4.28 Hyperparameters selected for MicroTEDAClus+ 50

Chapter 1

Introduction

Data streams arrive continuously, without a known end, from processes that can change while they are observed. Clustering them requires an algorithm that processes each sample as it arrives, keeps a bounded summary and follows the changes of the data, a family of methods known as stream or evolving clustering [1–3]. MicroTEDAClus [4] is one such algorithm. It summarizes the stream into microclusters by the eccentricity and the typicality of the TEDA framework [5], which assume no distribution for the data, and groups them into macroclusters, the clusters of the stream. Its authors report that it performs on par with established stream clustering algorithms, and that it adapts better than them to concept drift.

In this work, we reassess MicroTEDAClus and propose an improved model. We find issues in the formulation of the algorithm, differences between it and the implementation that produced the published results, and flaws in the methodology of the original experiments, among them a preprocessing that leaks information about the stream and an evaluation that predicts each batch after learning it. Under a methodology that corrects these flaws, MicroTEDAClus is not on par with the benchmark models of the original work, nor does it adapt better than them to concept drift. We obtain the improved model, MicroTEDAClus+, from controlled experiments on corrections and modifications of the algorithm. On the datasets of the original work it improves on MicroTEDAClus in clustering quality, execution time and memory, no benchmark model is better than it in the datasets with concept drift, and it is the fastest model of the experiment, while two benchmark models remain better than it in the static shapes.

1.1 Motivation

The results of MicroTEDAClus are the basis of later work. The doctoral thesis of Galindo [6] updates the algorithm and applies it to the last-mile problem in logistics, without examining the original evaluation. The undergraduate thesis of Cardoso [7] reimplements it in `C` to reduce its execution time, evaluates its quality only on static datasets that the model has already learned, and expects it to handle concept drift because the original work reports that it does. A model whose evaluation supports further work must be verified: information leaked from the data into the evaluation, among other errors, has been found to inflate the published results of many studies in machine learning [8, 9]. The algorithm is also attractive: the TEDA metrics require neither a distribution for the data nor a predefined number of clusters.

The TEDA framework is that of Angelov [5]. The formulation of MicroTEDAClus and its reference implementation are those of Maia et al. [4], and its updated formulation is that of Galindo [6]. Cardoso [7] had earlier observed that the outlier condition cannot hold for microclusters with two samples, and proposed an intersection condition with a factor. We execute the benchmark models through the `River` library [10] or, for CEDAS, TEDACloud and AutoCloud, through implementations that we made from their official code or papers. Our contribution is the reassessment of the formulation, the implementation and the methodology, the methodology that replaces it, the vectorized implementation, the experiments of the improvement and MicroTEDAClus+. The code and the sources of this thesis are public [11].

1.2 Objectives

Our objectives are:

a) to examine the formulation of MicroTEDAClus and its public implementation, and identify their issues and differences;

b) to examine the methodology of the experiments of the original work, and define one that evaluates every model alike and without its flaws;

c) to evaluate MicroTEDAClus against benchmark models under that methodology;

d) to propose an improved model, and evaluate it under the same methodology against MicroTEDAClus and the benchmark models.

Chapters 2 and 3 address the first three objectives, Chapter 4 the last, and Chapter 5 concludes the thesis.

Chapter 2

Background and Related Work

This chapter presents the eccentricity and the typicality of the TEDA framework (Section 2.1), the formulation of MicroTEDAClus by Maia et al. (Section 2.2), its update by Galindo (Section 2.3), and the reimplementation of the model in `C` by Cardoso (Section 2.4). After each formulation, we discuss its issues, and we point to where the following chapters examine them.

2.1 Typicality and Eccentricity-Based Data Analytics

Angelov [5] introduces the TEDA framework as an alternative to probability theory for data in non-deterministic settings, built on two metrics. The eccentricity ξ_j^k of a sample j at moment k, twice the sum π_j^k of its distances to all the samples seen up to k divided by the sum of these sums over all the samples, measures how much the sample deviates from the others, and its typicality $\tau_j^k = 1 - \xi_j^k$ how well it fits among them; both lie between zero and one once $k > 2$ and the sums are positive. Their normalized forms are

$$\zeta_j^k = \frac{\xi_j^k}{2}, \qquad t_j^k = \frac{\tau_j^k}{k-2}, \qquad \sum_{i=1}^{k} \zeta_i^k = \sum_{i=1}^{k} t_i^k = 1, \tag{2.1}$$

and a sample is anomalous if $\zeta_j^k > 1/k$ and typical if $t_j^k > 1/k$. For the squared Euclidean distance, the sums of the distances can be updated recursively from the mean and the mean squared norm of the samples.

Angelov also states that the metrics suit other proximity functions, among them the cosine similarity and the Mahalanobis distance [12].

2.2 Original MicroTEDAClus

Maia et al. [4] use the eccentricity and the typicality over the Euclidean distance to create and update microclusters, by detecting outliers and typical samples, and to merge the microclusters into macroclusters. They write the eccentricity in the recursive form

$$\xi(x_k) = \frac{1}{k} + \frac{(\mu_k - x_k)^T (\mu_k - x_k)}{k\sigma_k^2}, \tag{2.2}$$

with

$$\mu_k = \frac{k-1}{k}\mu_{k-1} + \frac{x_k}{k}, \qquad \sigma_k^2 = \frac{k-1}{k}\sigma_{k-1}^2 + \frac{1}{k-1}\|x_k - \mu_k\|^2, \tag{2.3}$$

and consider a sample an outlier if $\zeta(x_k) > (m^2+1)/(2k)$, with $m > 0$ a function of the number of samples chosen by the authors,

$$m(k) = \frac{3}{1 + e^{-0.007(k-100)}}. \tag{2.4}$$

A microcluster i at moment k is described by its number of samples S_k^i, the mean μ_k^i and the variance $(\sigma_k^i)^2$ of its samples, the eccentricity $\xi^i(x_k)$, the typicality $\tau^i(x_k)$ and their normalized forms $\zeta^i(x_k)$ and $t^i(x_k)$ of a sample x_k to it, its density $D_k^i = 1/\zeta^i(x_k)$, and its threshold function $m_k^i(S_k^i)$. The metrics apply to each microcluster as to the whole data. For $S_k^i > 2$, a sample x_k is an outlier to microcluster i if

$$\zeta^i(x_k) > \frac{m_k^i(S_k^i)^2 + 1}{2S_k^i}, \tag{2.5}$$

and, for $S_k^i = 2$, if

$$\left(\zeta^i(x_k) > \frac{m_k^i(2)^2 + 1}{4}\right) \wedge \left(\left(\sigma_k^i\right)^2 < r_0\right), \tag{2.6}$$

where r_0 is a hyperparameter, the variance limit. Every microcluster to which x_k is not an outlier is updated by

$$\begin{aligned} S_k^i &= S_{k-1}^i + 1, \qquad \mu_k^i = \frac{S_k^i - 1}{S_k^i}\mu_{k-1}^i + \frac{x_k}{S_k^i}, \\ \left(\sigma_k^i\right)^2 &= \frac{S_k^i - 1}{S_k^i}\left(\sigma_{k-1}^i\right)^2 + \frac{1}{S_k^i - 1}\left(\frac{2\|x_k - \mu_k^i\|}{d}\right)^2, \\ \xi^i(x_k) &= \frac{1}{S_k^i} + \frac{2\left(\mu_k^i - x_k\right)^T\left(\mu_k^i - x_k\right)}{S_k^i\left(\sigma_k^i\right)^2 d}, \end{aligned} \tag{2.7}$$

where d is the dimension of the data. A sample that is an outlier to every microcluster creates a new one, with one sample, its mean at the sample and no variance, as the first sample of the stream does.

The macroclusters are the connected components of the graph whose vertices are the microclusters and whose edges join the microclusters that satisfy

$$\operatorname{dist}(\mu_k^i, \mu_k^j) < 2\left(\sigma_k^i + \sigma_k^j\right), \quad i \neq j. \tag{2.8}$$

To keep overlapping microclusters from collapsing into a single macrocluster, a microcluster m_l^j takes part in the macrocluster $\mathcal{M}_j = \{m_1^j, \ldots, m_{N_j}^j\}$ only if it is active, that is, if

$$D_k^l \geq \operatorname{mean}\left(D_k^l\right), \quad l = 1, \ldots, N_j. \tag{2.9}$$

Each active microcluster weighs in its macrocluster by its share of the density of the active microclusters, and the typicality of a sample x_k to a macrocluster is the weighted sum of its typicalities to them,

$$\mathcal{T}_j(x_k) = \sum_{l \in \mathcal{M}_j} \frac{D_k^l}{\sum_{l' \in \mathcal{M}_j} D_k^{l'}} t_k^l(x_k), \tag{2.10}$$

the sample being assigned to the macrocluster of highest typicality. The formulation defines no decay of the microclusters, although the public implementation has one (Section 3.1).

2.2.1 Issues of the Original Formulation

Order-dependent density. The eccentricity and the typicality relate a sample to a microcluster, so the density $D_k^i = 1/\zeta^i(x_k)$ relates them too, and should not be treated as a property of the microcluster. A microcluster with a small variance is dense for any standard clustering algorithm [1, 2, 13], but if the current sample is far from its mean, $\zeta^i(x_k)$ is large and D_k^i small, and the microcluster is less likely to be activated. Conversely, a microcluster with a large variance and a sample close to its mean has a large D_k^i and is more likely to be activated, even if it is not dense at all. Section 4.3 tests two densities that do not depend on the current sample, and Section 4.4.4 tests both again on MicroTEDAClus+.

Outlier condition for microclusters with two samples. For $S_k^i = 2$, Equation (2.2) and the recursion of the mean and the variance give

$$\mu_2 = \frac{1}{2}(x_1 + x_2), \qquad \sigma_2^2 = \|x_2 - \mu_2\|^2, \qquad \xi(x_2) = 1, \qquad \zeta(x_2) = 0.5, \tag{2.11}$$

and, because $m \geq 1$,

$$\frac{m_k^i(2)^2 + 1}{4} \geq 0.5. \tag{2.12}$$

The first term of Equation (2.6) is therefore never satisfied, and neither is the conjunction: no sample can be an outlier to a microcluster with two samples. With the scaled expressions of Equation (2.7), the normalized eccentricity of the second sample becomes $1/4 + d/8$, so the first term is never satisfied with one or two dimensions and always satisfied with three or more, where the condition reduces to $(\sigma_k^i)^2 < r_0$, which holds for the samples close to the mean. Either way, the condition does not detect outliers. Replacing it by

$$(\sigma_k^i)^2 < r_0 \tag{2.13}$$

would keep the inversion, so the condition should be

$$(\sigma_k^i)^2 > r_0. \tag{2.14}$$

Cardoso [7] reached the same conclusion earlier, proving that the eccentricity of the second sample is always one, and detects the outliers of microclusters with fewer than three samples by Equation (2.14) alone, as the public implementation does (Section 3.1).

Dimensionality penalty. Equation (2.7) scales the variance and the eccentricity of a microcluster by the dimension d. No paper that introduces the TEDA metrics suggests such a scaling [5, 14], the original work does not justify it, and it lets the normalized eccentricity exceed its bound of one half. Section 3.8 traces the collapse of MicroTEDAClus in high dimensions to it, and Section 4.3 tests its removal.

2.3 Updated MicroTEDAClus

In the PhD thesis *Mineração de Dados Aplicada a Problemas de Last-Mile na Logística*, Galindo [6] updates MicroTEDAClus to apply it to the last-mile problem in logistics; we describe only the changes. It introduces an explicit decay: each microcluster has a vitality v_k^i, set to one when it is created and lowered by

$$v_k^i = v_{k-1}^i - \rho \tag{2.15}$$

whenever the sample x_k is an outlier to it, where $\rho = 1/\text{decay}$ and *decay* is a hyperparameter, and a microcluster is removed when $v_k^i \leq 0$. Section 4.4 tests this rule as the candidate *recency*. The update also corrects the direction of the comparison with r_0 in the condition for microclusters with two samples, but keeps its conjunction, and replaces the Euclidean distance by the cosine similarity, with the outlier condition that Angelov [14] derives for the Euclidean distance and a variance that remains Euclidean, neither of them justified for the new distance. The order-dependent density and the dimensionality penalty remain, and the rest of the algorithm is unchanged.

2.4 A C Implementation of MicroTEDAClus

In the undergraduate thesis *Implementação do algoritmo MicroTEDAclus para classificação de dados*, Cardoso [7] reimplements the original MicroTEDAClus in `C`, a language chosen for its speed and for its use in embedded platforms. The thesis documents the TEDA expressions that the model uses, with proofs and unit tests, and proposes an alternative to the intersection condition of Equation (2.8), which compares the distance between the means of two microclusters with the sum of their radii $m_k^i(S_k^i)\,\sigma_k^i$, scaled by a factor ϕ:

$$\text{dist}(\mu_k^i, \mu_k^j) < \phi \left(m_k^i(S_k^i)\,\sigma_k^i + m_k^j(S_k^j)\,\sigma_k^j \right), \quad \phi \geq 1, \quad i \neq j. \tag{2.16}$$

Its experiments evaluate the clustering quality by the adjusted Rand index on seven static datasets of two dimensions. The model learns each whole dataset before labeling it, in random orders of the samples and with hyperparameters taken from a grid, and the thesis reports the lowest and the highest index and the highest mean index over the configurations [7, Section 4.3]. It concludes that the quality depends strongly on r_0 and ϕ, and that Equation (2.16) is comparable to Equation (2.8). The reimplementation runs much faster than the `Python` implementation of the original work, measured over each whole program on a single machine and without repetitions [7, Section 4.4].

2.4.1 Issues of the Evaluation of the C Implementation

The highest indices belong to the configurations of the grid that best fit the labels of the evaluated data, and are therefore optimistic estimates of the quality that the model reaches when its hyperparameters are chosen without those labels. The lowest index

is no better than chance in every dataset [7, Tables 4.1 and 4.2], and the comparison between the two intersection conditions rests on the best cases, without a statistical test. The model is compared with no benchmark model, and none of the datasets has concept drift, which the thesis expects the model to handle because the original work reports that it does [7, Section 4.2].

Chapter 3

Methodology Reassessment

Maia et al. [4] report that MicroTEDAClus performs on par with established stream clustering algorithms and adapts better than them to concept drift. In this chapter, we examine what produced those results, from the implementation of the model to the evaluation protocol, define the methodology that the rest of the thesis uses, and evaluate MicroTEDAClus against six benchmark models under it.

3.1 Model Implementation

The original work points to a public repository with the code of its experiments, `https://github.com/cseveriano/evolving_clustering`, which implements MicroTEDAClus in `src/models/evolving/EvolvingClustering.py`. To preserve it, we forked the repository at its last commit, `50dd2b4` [15].

This implementation is not the formulation of the paper, but it is the code that produced the published results. It shares features with the updated formulation of Galindo [6], and it solves the outlier condition of microclusters with two samples (Section 2.2.1), but it departs from the formulation in the decay of the microclusters, in their activation, in the eccentricity and in the membership.

Decay. The constructor turns the hyperparameter `decay` into `fading_factor` = 1/decay, the rate ρ of the decay of Galindo (Equation (2.15)). The factor acts in two places. The pruning (Listing 3.1) lowers, at every update, the vitality of the microclusters that are not active, while Galindo lowers that of the microclusters that the current sample does not update; activation is decided only when the macroclusters are built. The vitality update (Listing 3.2) rewards a microcluster when a sample falls within one standard deviation of its mean and penalizes it otherwise, a rule that appears in neither work and that neither explains.

```
def prune_micro_clusters(self):
    for mc in self.micro_clusters:
        if not mc["active"]:
            mc["life"] = mc["life"] - self.fading_factor
            if mc["life"] < 0:
                self.micro_clusters.remove(mc)
                if mc in self.changed_micro_clusters:
                    self.changed_micro_clusters.remove(mc)
                self.graph.remove_node(mc["id"])
```

Listing 3.1: Pruning of the microclusters in the public implementation of MicroTEDAClus

```
def update_life(self, xk, micro_cluster):
    previous_mean = micro_cluster["mean"]
    previous_var = micro_cluster["variance"]
    if previous_var > 0:
        d = EvolvingClustering.get_euclidean_distance(xk, previous_mean)
        dist = np.sqrt(np.sum(d))
        rt = np.sqrt(previous_var)
        micro_cluster["life"] = micro_cluster["life"] + (((rt - dist) / rt)
    * self.fading_factor)
    else:
        micro_cluster["life"] = 1
```

Listing 3.2: Update of the vitality of a microcluster in the public implementation of MicroTEDAClus

Other departures. The implementation departs from the paper in four more places, all in the file of the model in the forked repository [15]. It builds the macroclusters in two passes: it groups the intersecting microclusters into the components of a graph $\mathbb{G}_0$, activates microclusters within each component by Equation (2.9), removes the inactive ones to obtain a graph $\mathbb{G}_1$, and returns the components of $\mathbb{G}_1$ as the macroclusters, which guards against the collapse of overlapping microclusters into a single macrocluster that the original work anticipates (Section 2.2). Only microclusters with more than two samples can be active, while the mean density is computed over all the microclusters of the component. Its outlier test applies Equation (2.5) from three samples on and, below three, the test of the variance alone of Equation (2.14). Its eccentricity differs from Equation (2.2) by a factor $4/d^2$ on the second term. Its membership weighs $1 - \zeta$ by the densities of the active microclusters, an expression that cannot be rewritten as the normalized typicality of Equation (2.1).

The implementation of the updated formulation, which Galindo provided to us, is not public. It adds partial support for the cosine similarity, refactors the decay and

moves the microclusters into a class of their own.

3.2 Benchmark Models

The original work compares MicroTEDAClus with DenStream [2], CluStream [1] and StreamKM ++ [16], benchmarks common in the literature on stream clustering [3, 17–21]. It executes them from `Python` through `rpy2`, which calls the `R` package `streamMOA`, itself a wrapper of the `Java` framework MOA [22]. These two layers of interface add to the time and memory that the experiment measures. To quantify them, we timed each model and measured its memory on a stream of the Random RBF generator with 100,000 samples, 10 dimensions, five classes and 50 clusters, eight times, through the bridge and directly in `Java`, with the settings of `streamMOA`: CluStream with 100 microclusters and five macroclusters, DenStream with $\epsilon = 0.02$, $\mu = 1$, $\beta = 0.2$, $\lambda = 0.001$, 100 initial points and an offline factor of 2, and StreamKM with a coreset of 10,000 samples and five clusters. Through the bridge, every model takes much longer, while its memory barely changes (Figure 3.1). The comparison of the original work therefore measures the interfaces together with the models, and a fair comparison requires every model to run in the environment of the evaluated one.

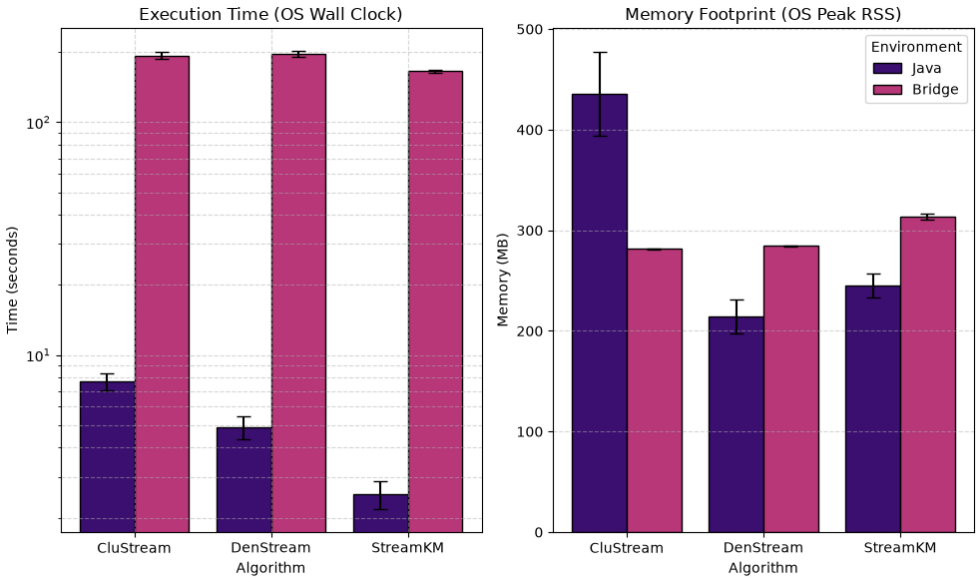


Figure 3.1: Execution time, at left, and memory, at right, of the benchmark models of the original work through the `rpy2` bridge and directly in `Java`.

Since MicroTEDAClus is implemented in `Python`, we use the implementations of DenStream, CluStream and StreamKM in the `River` library [10].

3.3 Datasets

The original work evaluates the models on static datasets, streams with concept drift and scalability streams of increasing dimension, common in the literature [23–27], but the files of the original work hide preprocessing decisions that change the results and that a stream does not allow. Table 3.1 lists the datasets that we use instead, with the abbreviations of the tables.

Table 3.1: The datasets of the main experiment and their abbreviations.

Dataset	Abbreviation	Kind
StreamBenchmark(1)	B1	Concept drift
StreamBenchmark(2)	B2	Concept drift
RandomRBFGeneratorEvents	RBF	Concept drift
Cassini	CSN	Static
DS3	DS3	Static
DS4	DS4	Static
SCL-1 to SCL-64	S1 to S64	Scalability, of 1 to 64 dimensions

3.3.1 Static Datasets

The static datasets ST-D1 and ST-D2 of the original work come from the evaluation of the Chameleon algorithm [28], whose paper names the raw patterns, of 10,000 and 8,000 samples, DS3 and DS4. We use the raw files `cluto-t7-10k.arff` and `cluto-t8-8k.arff`, and those names. The files of the original repository, `chameleon-ds3-clean.csv` and `chameleon-ds4-clean.csv`, are cleaned of noise, downsampled, scaled and ordered by label (Figure 3.2).

Removing the samples labeled as noise is a severe leak [8, 9]: a sample is noise only if it belongs to no cluster, which is known after the clustering, and in a stream the model itself must decide which samples are noise, from the samples it has seen, or not handle noise at all [3, 18–21, 25, 27]. Keeping a random half of the samples needs no information about them and is not a leak. Scaling is expected in any application, but the original work computes the minimum and the maximum of each feature over the whole dataset, which leaks the future of the stream. The variance limit that the search selects also follows the scale of the data (Table 3.5), against the claim of the original work that the model is robust to it, and the sensitivity study of Section 2.4 finds the quality strongly dependent on it. Ordering the samples by label is the most harmful step: it leaks the labels and discards the order of arrival, and, since consecutive samples then share their label, most 100-sample batches hold a single cluster. Together, the

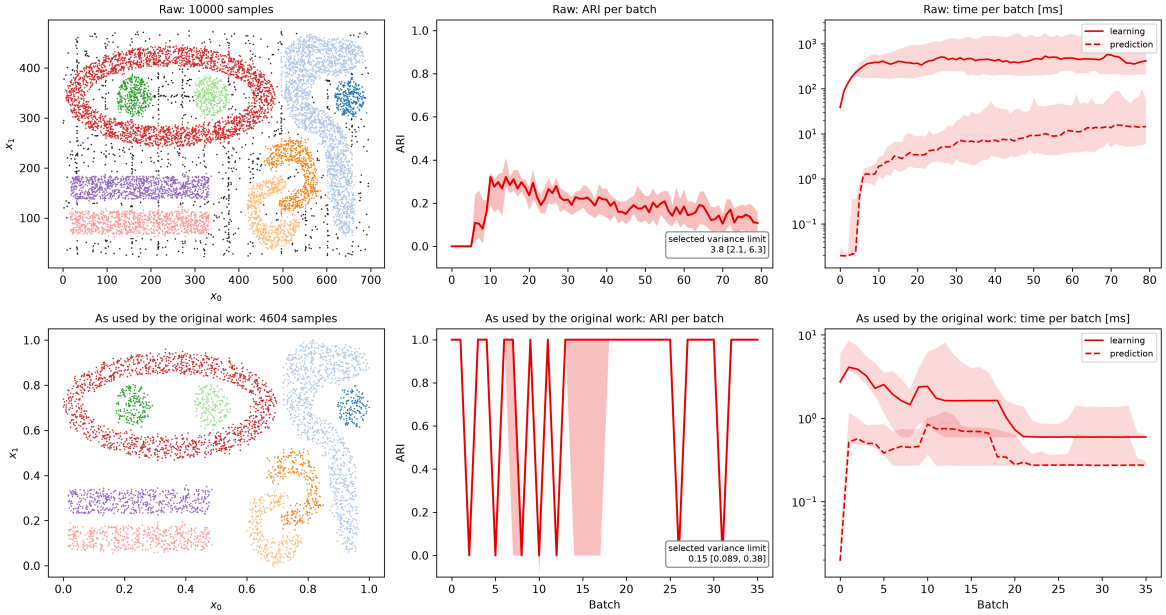


Figure 3.2: DS3 raw, at top, and as the original work used it, after the noise removal, the downsampling to half of its samples, the min-max scaling and the ordering by label, at bottom: its samples, with the noise in black, and the ARI and the learning and prediction times per batch of MicroTEDAClus tuned by the protocol of this chapter, with the variance limit that the search selected. Medians and interquartile bands over 94 runs.

steps raise the ARI of MicroTEDAClus to one in most batches, cut its learning time by about two orders of magnitude and its prediction time by one, and lead the search to a variance limit more than an order of magnitude smaller (Figure 3.2).

The experiments of the original work on ST-D1 and ST-D2 therefore do not evaluate online stream clustering, and we use the raw DS3 and DS4 instead. The third static dataset, Cassini, comes from the `R` package `mlbench` without preprocessing, but ordered by label; we shuffle it.

3.3.2 Concept Drift Datasets

The streams with concept drift of the original work, StreamBenchmark(1), StreamBenchmark(2) and RandomRBFGeneratorEvents, come from `streamMOA`. Its plots suggest that their noise samples, labeled `NA` by the package, were removed. We keep them, labeled -1.

3.3.3 Scalability Streams

The scalability experiments of the original work, which profile memory and time as the number of samples and the dimension grow, use synthetic streams of `streamMOA`

without noise. We generate them with noise.

3.4 Evaluation Protocol

The original work states that it applies prequential evaluation, in which "each instance can be used to test before it is used for training" [4, Section 5.2.1], the standard practice for stream clustering [3, 18–21], in line with Dawid's view of inference as forecasting future observations [29]. Its code, however, implements no prequential evaluation.

For ST-D1 and ST-D2, it learns the whole dataset and then predicts the same samples (Listing 3.3). Even for offline clustering, that is not standard practice [8, 9, 30, 31], which prefers cross-validation [32, 33]. The evaluation of the `C` reimplementation of Section 2.4 follows the same protocol for all of its datasets. For the other datasets, a function named `prequential_evaluation` learns each batch before predicting it (Listing 3.4), the opposite of prequential evaluation.

Every experiment of this thesis uses prequential evaluation: each 100-sample batch is predicted before the model learns it, and the predicted labels are compared with the true ones.

```
X = stream_df[['x', 'y']].values
y = stream_df['class'].values
evol_model = EvolvingClustering.EvolvingClustering(variance_limit=0.0006,
    debug=False)
evol_model.fit(X)
y_hat = evol_model.predict(X)
error = adjusted_rand_score(y, y_hat)
```

Listing 3.3: Evaluation of the static datasets in the original work

```
if elapsed_time:
    time_start = time.time()
method.fit(train_data)
y_hat = method.predict(test_data)
y = labels[test_start:test_end]
```

Listing 3.4: Excerpt of the function named `prequential_evaluation` in the original work

3.5 Metrics

The original work evaluates clustering quality by the adjusted Rand index (ARI) [34], an external index that depends neither on the distribution of the data nor on the number of clusters [31]. It summarizes the ARI of a run by its mean and standard deviation over the runs, which hides how the quality evolves along a stream: a curve that improves slowly, one that peaks and degrades, and a random one can share both. We summarize the ARI of a run by two metrics:

- the **Overall ARI**, the median of the ARI of the batches of the evaluation segment;
- the **Steady-State ARI**, the ARI of the penultimate batch, which measures the quality after the stream has run its course and any concept drift has occurred. The last batch is avoided because it may be incomplete.

The original work reports the execution time only for its scalability streams, as the mean time per sample against the dimension, and the memory only for one run of StreamBenchmark(1). We measure the cost of every model on every dataset:

- the **execution time** of the learning and of the prediction stages, each summed over the batches of the evaluation segment, from the wall clock inside the task. Section 4.2.2 measures the time with each task executed alone;
- the **memory footprint**, the size of the model object after each batch, at its maximum and at the penultimate batch;
- the **microcluster count**, at its maximum and at the penultimate batch, a proxy for the memory that depends less on the implementation. StreamKM keeps no microclusters and is left out of it.

For the scalability streams, we report the quality per dimension and plot the cost against the dimension.

3.6 Hyperparameter Selection and Runs

3.6.1 Hyperparameter Tuning

So that no model is disadvantaged by its hyperparameters, even those whose papers use default values, every run begins with a search on the first 20% of its stream, by Bayesian optimization [35], and the tuned model is evaluated on the remaining 80%, which the search never sees [8,9]. The original work, in contrast, selects the parameters that maximize the score over the whole dataset. The search maximizes the objective of Algorithm 1: the ARI of a prequential execution on the tuning segment, without its first 5% of predictions, less a penalty of $\lambda = 10^{-3}$ per second of execution.

Algorithm 1 Objective of the hyperparameter search

Require: the tuning segment (X_{opt}, y_{opt}), the batch size, the model class and its search space, the current trial
Ensure: the score that the search maximizes
1: Draw the hyperparameters of the trial from the search space
2: Build the model with them
3: Execute it prequentially on the tuning segment, measuring the execution time t_{total}
4: Drop the first 5% of the predictions
5: Compute the ARI ari between the remaining predictions and their labels
6: **return** $ari - \lambda\, t_{total}$

The search of each model ends at 100 trials or at 300 seconds, whichever comes first. The penalty and the time limit avoid configurations that buy a small gain in ARI with a large cost, and they keep the experiments feasible: some models are slow enough to make an unrestricted search impossible over the runs that the statistics require, so a slow model may complete fewer trials than a fast one. For the same reason, every dataset has at most 10,000 samples. Section 4.6 examines the limits of this search.

3.6.2 Multi-run Statistics

A run generates its stream from its seed, tunes the model and evaluates it. We aggregate the runs by the median and the interquartile range, and not by the mean and the standard deviation, because the search may end in a poor local optimum and produce outlier runs, to which the median is robust. We choose the number of runs from 1000 runs of SCL-2, StreamBenchmark(1) and StreamBenchmark(2), in the ARI, the time, the memory and the microcluster count: for each number of runs k, we draw 10^4 subsets of k runs and measure the mean absolute error of their median against the median of the 1000 runs, normalized by its interquartile range. The number of runs is the knee

of the worst of these curves, beyond which more runs bring diminishing returns, as the Kneedle algorithm [36] finds it: $k^* = 94$.

3.7 Statistical Comparison

Before any test, we discretize the metrics to thresholds of practical relevance, the ARI to 10^{-3}, the times to 10 ms, the memory to 1 KiB and the microcluster count to integers, so that floating-point noise and hardware jitter do not inflate the power of the rank-based tests. Every comparison sets one model against references [37]: the model in focus against each other model, wherever a model is compared with the benchmark models, and a variant against its control, in the improvement of Chapter 4. In each dataset and metric, the paired Wilcoxon signed-rank test [38] compares them over the runs that both finished, and the Holm–Bonferroni procedure [39] adjusts the p-values of the comparisons of a model in focus, or of the datasets of a variant, together. With 94 runs, the test rejects the null hypothesis for differences of no practical consequence, so a model is distinguishable from its reference only if the adjusted p-value is below $\alpha = 0.05$ and Cliff's delta [40] reaches $|d| \geq 0.147$, the boundary of a non-negligible effect [41].

Reading the tables. The tables give medians over the runs. In a comparison with the benchmark models, the model in focus comes first, followed by a row that counts the models it is significantly better than, indistinguishable from and worse than. The median of another model is in bold where the model in focus is significantly better than it, and in red where the model in focus is significantly worse. In the tables of a variant and its control, the value of the variant is in bold where it is significantly better than its control, and in red where it is significantly worse. Higher is better for the ARI; lower is better for the execution time, the memory and the size of a summary.

3.8 MicroTEDAClus Under the Reassessed Methodology

We evaluate MicroTEDAClus against the six benchmark models of Section 3.2 on the six datasets and the seven scalability streams of Section 3.3, over 94 runs, under the methodology of the previous sections.

Table 3.2: Clustering quality of MicroTEDAClus and the benchmark models: medians over the runs of the main experiment.

Model	B1	B2	RBF	CSN	DS3	DS4
Overall ARI						
MicroTEDAClus	0.035	0.564	0.359	0.492	0.215	0.202
Better/same/worse	0/3/3	5/1/0	2/3/1	3/1/2	4/0/2	3/1/2
DenStream	0.665	**0.402**	0.360	0.383	**0.174**	0.184
CluStream	0.096	0.566	**0.170**	0.573	0.324	0.319
StreamKM	0.183	**0.498**	0.371	0.696	0.330	0.327
AutoCloud	0.043	**0.001**	**0.051**	**0.076**	**0.023**	**0.030**
CEDAS	0.252	**0.000**	0.178	**0.000**	**0.000**	**0.000**
TEDACloud	0.827	**0.509**	0.495	**0.360**	**0.040**	**0.125**
Steady-State ARI						
MicroTEDAClus	0.000	0.524	0.000	0.436	0.126	0.124
Better/same/worse	0/1/5	2/2/2	0/3/3	2/2/2	2/1/3	2/1/3
DenStream	0.822	0.567	0.340	0.405	0.196	0.274
CluStream	0.174	0.599	0.194	0.589	0.338	0.320
StreamKM	0.164	0.860	0.451	0.569	0.341	0.301
AutoCloud	0.020	**0.135**	0.040	**0.052**	**0.073**	**0.070**
CEDAS	0.211	**0.000**	0.142	**0.000**	**0.000**	**0.000**
TEDACloud	0.760	0.518	0.541	0.366	0.117	0.091

3.8.1 Clustering Quality

The results do not support the claims of the original work (Table 3.2). Against its benchmark models, MicroTEDAClus is not on par: in the Overall ARI, it is worse than CluStream and StreamKM in the three static datasets and than DenStream in StreamBenchmark(1), and better than each of the three in at most two datasets. Nor does it adapt better to concept drift. At the penultimate batch, it is better than none of the three in any dataset and worse than StreamKM in all six. In StreamBenchmark(1) and RandomRBFGeneratorEvents, two of the streams with concept drift, its median Steady-State ARI is zero, and it is worse than five and three of the six benchmark models, respectively. In the Overall ARI, it is better than most benchmark models only in StreamBenchmark(2) and DS3, mainly the TEDA-based models AutoCloud, CEDAS and TEDACloud, which it beats in every static dataset.

In the scalability streams, the Overall ARI of MicroTEDAClus stays high at every dimension, but its Steady-State ARI falls with 32 dimensions and reaches zero with 64 (Table 3.3): its quality collapses before the end of the stream (Figure 4.2), which we trace to the dimensionality penalty of Section 2.2.1. From eight dimensions on, it is worse than CluStream, StreamKM and CEDAS in both metrics.

Table 3.3: Clustering quality of MicroTEDAClus and the benchmark models in the scalability streams: medians over the runs of the main experiment.

Model	S1	S2	S4	S8	S16	S32	S64
Overall ARI							
MicroTEDAClus	0.763	0.861	0.860	0.867	0.883	0.865	0.843
Better/same/worse	4/2/0	3/3/0	2/2/2	2/1/3	2/1/3	2/1/3	2/1/3
DenStream	**0.000**	**0.690**	**0.442**	**0.426**	**0.000**	**0.000**	**0.000**
CluStream	0.661	0.887	0.895	0.913	0.953	0.980	0.991
StreamKM	0.665	0.887	0.927	0.953	0.971	0.983	0.989
AutoCloud	**0.325**	**0.446**	**0.405**	**0.426**	**0.435**	**0.443**	**0.445**
CEDAS	**0.511**	0.932	1.000	1.000	1.000	1.000	1.000
TEDACloud	**0.495**	**0.536**	0.565	0.924	0.567	0.503	0.586
Steady-State ARI							
MicroTEDAClus	0.727	0.857	0.856	0.847	0.810	0.462	0.000
Better/same/worse	4/2/0	3/2/1	2/1/3	2/1/3	2/1/3	2/1/3	1/0/5
DenStream	**0.000**	**0.768**	**0.493**	**0.454**	**0.000**	**0.000**	**0.000**
CluStream	0.677	0.873	0.894	0.916	0.960	0.980	0.991
StreamKM	0.621	0.903	0.934	0.957	0.972	0.985	0.990
AutoCloud	**0.280**	**0.419**	**0.372**	**0.379**	**0.421**	**0.419**	0.433
CEDAS	**0.508**	0.885	1.000	1.000	1.000	1.000	1.000
TEDACloud	**0.532**	**0.583**	0.606	0.903	0.598	0.529	0.510

3.8.2 Cost

MicroTEDAClus is slower than every benchmark model but AutoCloud in the learning stage of every dataset, and slower than StreamKM and CEDAS in the prediction stage of every dataset (Table 3.4). Its maximum memory footprint is larger than those of DenStream, StreamKM, CEDAS and TEDACloud in every dataset, and larger than that of CluStream in the Chameleon datasets. These comparisons measure implementations as much as algorithms, and the times are taken inside the tasks, with other tasks running on the same machine.

3.8.3 Hyperparameters

The variance limit selected for MicroTEDAClus spans four orders of magnitude across the datasets, following the scale of each of them (Table 3.5), which contradicts the claim of the original work that the model is robust to this hyperparameter. The dispersion of the selected decay is higher in the three streams with concept drift than in the static datasets.

Table 3.4: Execution time and memory footprint of MicroTEDAClus and the benchmark models: medians over the runs of the main experiment.

Model	B1	B2	RBF	CSN	DS3	DS4
Learning time [ms]						
MicroTEDAClus	1000	570	8210	590	27850	18815
Better/same/worse	0/1/5	0/1/5	0/1/5	1/0/5	0/0/6	1/0/5
DenStream	180	200	200	190	210	210
CluStream	230	140	4260	120	380	375
StreamKM	60	30	120	25	120	105
AutoCloud	12910	50	460	**16910**	445	**97150**
CEDAS	30	20	90	30	90	70
TEDACloud	110	50	530	270	360	310
Prediction time [ms]						
MicroTEDAClus	100	50	800	60	480	445
Better/same/worse	1/2/3	2/0/4	0/2/4	3/1/2	0/0/6	1/0/5
DenStream	90	**140**	150	**135**	70	110
CluStream	100	**80**	1220	**80**	260	200
StreamKM	10	10	30	10	30	30
AutoCloud	**4770**	20	160	**6200**	160	**35680**
CEDAS	20	10	60	20	70	50
TEDACloud	40	20	150	70	110	100
Maximum memory footprint [KiB]						
MicroTEDAClus	277	141	1030	140	702	628
Better/same/worse	0/2/4	0/2/4	0/2/4	0/2/4	0/0/6	1/0/5
DenStream	15	28	10	24	10	10
CluStream	209	152	1066	148	218	201
StreamKM	27	28	84	31	35	38
AutoCloud	654	3	11	706	9	**1941**
CEDAS	8	3	9	5	8	8
TEDACloud	6	3	11	12	7	7
Steady-state memory footprint [KiB]						
MicroTEDAClus	155	106	423	118	698	626
Better/same/worse	0/2/4	1/1/4	0/2/4	1/1/4	0/0/6	1/0/5
DenStream	12	23	9	18	6	9
CluStream	209	**152**	1066	**148**	218	201
StreamKM	16	17	31	17	20	28
AutoCloud	629	2	9	685	6	**1914**
CEDAS	7	3	8	5	8	8
TEDACloud	6	2	9	8	3	4

Table 3.5: Hyperparameters selected for MicroTEDAClus: median [QCD] over the runs of the main experiment.

Model	Hyperparameter	B1	B2	RBF	CSN	DS3	DS4
MicroTEDAClus	variance limit	0.00602 [0.93]	0.00838 [0.98]	0.00093 [0.59]	0.105 [0.44]	4.86 [0.23]	6.86 [0.59]
	decay	363 [0.87]	32.5 [0.87]	1040 [0.94]	35 [0.57]	422 [0.51]	367 [0.54]

3.9 Summary

The published results of MicroTEDAClus come from an implementation that departs from its formulation, benchmark models executed through two layers of interface, datasets cleaned, scaled with the whole stream and ordered by label, and an evaluation that learns each batch before predicting it. Without these flaws, MicroTEDAClus is not on par with the benchmark models of the original work, it does not adapt better to concept drift, it learns slowly, its quality collapses in high dimensions, and its variance limit depends on the scale of the data (Table 3.5). Chapter 4 addresses these weaknesses.

Chapter 4

Model Improvement

Chapters 2 and 3 identify issues in the formulation of MicroTEDAClus (Section 2.2.1) and undocumented behaviors of its public implementation (Section 3.1). In this chapter, we act on them in three stages. We first optimize the implementation, whose execution time made iterative experimentation unaffordable. We then evaluate corrections of the flaws, and finally modifications beyond the original formulation, which define MicroTEDAClus+. We close the chapter by comparing MicroTEDAClus+ with MicroTEDAClus and the benchmark models in the main experiment (Section 4.8).

4.1 Experimental Design

4.1.1 Variants and Controls

An experiment compares a *variant* of the model with its *control*. The variant differs from the control by a stated combination of changes, so any difference between their results is attributable to those changes. To make this exact, we implement every variant as a configuration of a single implementation, in which each design choice under study is a switch. With every switch in its default position, that implementation executes the original algorithm.

4.1.2 Protocol and Datasets

We tune and evaluate every variant by the protocol of the main experiment, with the metrics of Section 3.5. We use eight streams: the six datasets of the main experiment and its two scalability streams of highest dimension, SCL-32 and SCL-64, because the six datasets are two-dimensional and several properties of the model only show in higher dimensions. The tables use the abbreviations of Table 3.1.

4.1.3 Development and Validation Runs

A run is identified by an index, which determines the seed of the stream generator and of the hyperparameter search. The main experiment uses the runs 0 to 93. In this chapter, we use the runs that follow them, so the streams of the main experiment take no part in the development of the model:

- **Development runs** (94 to 125, 32 runs): we execute every experiment that compares variants on these runs, and we made every design choice of this chapter from their results.

- **Validation runs** (126 to 219, 94 runs): we evaluate again on these runs the models accepted during development, and only those.

A model chosen for scoring best on a set of runs is expected to score somewhat worse on others. The validation runs come from seeds that no development run uses, and we executed them once, after fixing the accepted models, so their streams take no part in the choices they measure; their number is that of Section 3.6.2. The Chameleon datasets are an exception: they are fixed files, so their runs differ only in the seed of the search, and the validation repeats the search and not the stream.

4.1.4 Comparisons and Decision Rule

We compare a variant with its control in each dataset and metric, over the runs they share, by the test of Section 3.7, whose conventions the tables follow, and decide the outcome on two metrics, the Overall and the Steady-State ARI. A variant

- **improves** its control if it is significantly better in at least one dataset and significantly worse in none;

- **regresses** if it is significantly worse in at least one dataset and significantly better in none;

- is **mixed** if both happen, and **indistinguishable** if neither does.

Only a variant that improves its control can replace it. The microcluster count and the memory footprint never decide an outcome.

4.2 Implementation Optimization

The reference implementation stores each microcluster as a dictionary in a Python list. For every sample, it iterates over that list and updates one microcluster at a time; it maintains the intersection graph edge by edge in a graph library, and it computes the membership of each sample to each macrocluster in nested loops. Its execution time is dominated by the interpreter, not by the arithmetic of the algorithm. The reimplementation of Section 2.4 reduces it by changing the language. We keep `Python`, the language of the benchmark models, and change how the algorithm is computed.

The optimized implementation, hereafter the *vectorized* model, stores the population of microclusters as a structure of arrays, with one array per attribute. The algorithm has a part that depends on the order of arrival of the samples: the recursive update of the microclusters, the outlier test, the vitality update and the pruning. This part cannot be vectorized across samples, so it runs in a single compiled kernel that iterates over the samples of a batch and, for each of them, over the arrays. Everything else is expressed as array operations. At each macrocluster update, the intersection graph is rebuilt from the matrix of pairwise distances between the microcluster means, its connected components are extracted by a sparse graph routine, and the membership of a whole batch is obtained from one distance matrix.

The *vectorized* model executes the algorithm of the reference implementation without reproducing its output exactly. Besides the order of the floating-point operations, the pruning routine of the reference implementation (Listing 3.1) removes elements from the list it iterates over, so it skips the microcluster that follows a removed one, while the *vectorized* model visits every microcluster. Each design choice studied in the rest of this chapter is an option of this implementation, whose default options are the *vectorized* model.

4.2.1 Clustering Quality

Table 4.1 compares the *vectorized* model with the reference implementation, the *original* model of the tables of this chapter, each tuned and evaluated by the protocol on the development runs. By the decision rule, the *vectorized* model improves the *original* one: its Overall ARI is significantly higher in DS3, and it is not significantly lower in any dataset on either metric. The remaining medians differ slightly.

The two models do not receive the same search: the reference implementation exhausts the 300 seconds before its 100 trials in RandomRBFGeneratorEvents, DS3 and DS4 (Table 4.2), so in DS3, where the *vectorized* model gains, the latter is tuned with several times the trials. The experiment does not separate this difference of budget

Table 4.1: Implementation optimization: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
original	0.000	0.532	0.335	0.457	0.206	0.220	0.869	0.857
vectorized	0.000	0.581	0.328	0.520	**0.230**	0.235	0.851	0.859
Steady-State ARI								
original	0.000	0.480	0.000	0.400	0.171	0.171	0.000	0.000
vectorized	0.000	0.541	0.000	0.458	0.176	0.209	0.000	0.000

Table 4.2: Hyperparameter search of the two implementations: medians over the development runs. The search ends at 100 trials or 300 seconds.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Completed trials								
original	100	100	64	100	17	32	100	100
vectorized	100	100	100	100	100	100	100	100
Duration of the search [s]								
original	29	37	302	89	303	303	72	81
vectorized	2	2	7	3	10	9	4	3

from that of the pruning: the speed of an implementation is also a search budget.

The optimization therefore preserves the clustering quality of the reference implementation, and the *vectorized* model replaces it as the reference of the remaining experiments.

4.2.2 Execution Time

The durations of Table 4.2 do not measure the optimization alone. They are taken while other tasks run on the same machine, and the two models are tuned independently, so a difference between them mixes the implementation, the load of the machine and the hyperparameters that each search selected. We therefore measure the speedup separately.

Each implementation replays the evaluation segment of every development run with the same hyperparameters: those tuned for the *original* model in that run and, in a second measurement, those tuned for the *vectorized* model. We execute each task alone, on identical processor cores, and run the compiled kernel once before timing it. We execute the whole set of tasks three times, and the time of a task is the minimum of its three executions.

Table 4.3 verifies the speedup in both measurements, so it does not depend on

Table 4.3: Execution time of the evaluation segment, learning plus prediction, of the two implementations with the same hyperparameters, and speedup of the vectorized one: median and quartiles over the 32 development runs. Each task runs alone, and its time is the smallest of three executions, which differ by 0.8% in the median.

Dataset	*original* [ms]	*vectorized* [ms]	Speedup
Hyperparameters tuned for the original model			
B1	613 (309, 3906)	35.1 (29.2, 90.9)	17.5 (10.6, 43.4)
B2	1080 (191, 1562)	40.7 (26.2, 47.3)	26.2 (7.3, 33.0)
RBF	17534 (4898, 37424)	300 (134, 612)	55.9 (35.6, 64.3)
CSN	1229 (1008, 1543)	43.2 (39.3, 46.0)	28.5 (25.2, 34.0)
DS3	9628 (9504, 23961)	178 (164, 347)	58.7 (57.2, 68.2)
DS4	9669 (8090, 16455)	166 (144, 236)	59.0 (56.7, 68.7)
S32	780 (445, 1858)	41.2 (34.2, 61.1)	18.9 (12.9, 29.6)
S64	828 (498, 1915)	45.4 (38.0, 68.0)	18.1 (13.1, 25.4)
All	2074 (732, 9510)	65.2 (39.3, 166)	33.1 (16.7, 57.4)
Hyperparameters tuned for the vectorized model			
B1	1092 (461, 2808)	43.4 (32.3, 71.9)	25.2 (14.3, 39.2)
B2	1126 (192, 1485)	42.2 (26.3, 46.3)	26.6 (7.3, 32.8)
RBF	14262 (2907, 30895)	261 (106, 524)	54.8 (27.8, 63.8)
CSN	1233 (1054, 1358)	42.8 (40.2, 46.2)	28.5 (26.2, 30.6)
DS3	9488 (9445, 9557)	166 (163, 176)	57.6 (56.1, 58.9)
DS4	9646 (8031, 15936)	165 (142, 230)	58.4 (57.0, 63.3)
S32	838 (471, 2147)	41.7 (34.7, 65.9)	20.1 (13.6, 33.9)
S64	972 (653, 2443)	47.9 (41.6, 91.7)	19.9 (15.6, 23.6)
All	2291 (919, 9447)	69.2 (41.6, 163)	32.5 (20.9, 57.0)

which model selected the configuration, and the *vectorized* model is at least twice as fast as the *original* one in every run.

The reduction is what makes the remaining stages affordable: the experiments of this chapter execute more than sixteen thousand tuning and evaluation tasks, each with up to 100 trials.

4.3 Corrections

The candidate corrections come from the issues raised in the previous chapters and from two behaviors of the reference implementation that we noticed while reimplementing it. The incorrect condition for microclusters with two samples, Section 2.2.1, is not among them, because the reference implementation already applies Equation (2.14).

Table 4.4 lists the candidates: five memberships that replace that of the implementation (Section 3.1), two densities that do not depend on the current sample (Section 2.2.1), two changes of the activation and the removal of the dimensionality penalty

(Section 2.2.1).

Table 4.4: Candidate corrections, each a change of the *vectorized* model; q is the standardized squared distance of Equation (4.2).

Variant	Change
norm-typicality	membership by the implemented weights with the normalized typicality of Equation (2.10)
local-weights	membership by Equation (2.10) with each density evaluated at the sample, $D^l(x) = 1/\zeta^l(x)$
density-sum	membership by the sum of the densities at the sample, $\sum_l 2S^l/(1+q^l(x))$
density-max	membership by the largest density at the sample
nearest	membership by the smallest standardized distance q to an active microcluster
peak-density	density of a microcluster at its mean, $2S$, instead of the density of its last sample
volume-density	density $S/(\sigma^2+\epsilon)$
mature-mean	mean density of the activation computed over the microclusters with three samples or more
born-inactive	new microclusters created inactive, so that they decay from the start
no-penalty	factor $2/d$ of Equation (2.7) removed

4.3.1 Corrections One at a Time

Table 4.5 compares each candidate with the *vectorized* model. Two candidates improve it: *nearest*, in the Overall ARI of RandomRBFGeneratorEvents and SCL-32 and in the Steady-State ARI of RandomRBFGeneratorEvents, Cassini, SCL-32 and SCL-64, and *no-penalty*, in the Steady-State ARI of SCL-32 and SCL-64.

Among the memberships, only the count-free distance helps. *density-max* is indistinguishable from the implemented membership and *density-sum* is worse. The two memberships built on the normalized typicality of the formulation, *norm-typicality* and *local-weights*, gain in the Steady-State ARI of DS4 and SCL-64 and lose in the Overall ARI of DS3, so the formulation is not preferable to its implementation in this respect.

Neither order-independent density improves the model. *peak-density* is worse in DS3 on both metrics, and *volume-density* trades a gain in the Steady-State ARI of RandomRBFGeneratorEvents for a loss in the Overall ARI of Cassini. The order dependence is real, but in this algorithm the density only decides which microclusters are active and how they are weighted, and for those purposes the alternatives are not better. *mature-mean* and *born-inactive* also regress, in DS3 and DS4 respectively.

Table 4.5: Corrections, one at a time: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
vectorized	0.000	0.581	0.328	0.520	0.230	0.235	0.851	0.859
born-inactive	0.133	0.550	0.410	0.489	0.216	0.207	0.893	0.863
peak-density	0.000	0.583	0.266	0.478	0.214	0.257	0.854	0.859
volume-density	0.004	0.559	0.402	0.319	0.228	0.223	0.817	0.804
mature-mean	0.258	0.553	0.460	0.486	0.231	0.259	0.842	0.858
no-penalty	0.000	0.576	0.328	0.512	0.232	0.256	0.864	0.860
norm-typicality	0.000	0.555	0.399	0.482	0.202	0.219	0.821	0.845
local-weights	0.000	0.562	0.472	0.478	0.194	0.223	0.839	0.861
density-sum	0.000	0.507	0.356	0.487	0.231	0.265	0.861	0.851
density-max	0.000	0.554	0.345	0.518	0.231	0.239	0.861	0.855
nearest	0.037	0.578	**0.498**	0.599	0.239	0.242	**0.964**	0.856
Steady-State ARI								
vectorized	0.000	0.541	0.000	0.458	0.176	0.209	0.000	0.000
born-inactive	0.000	0.528	0.002	0.266	0.177	0.146	0.684	0.000
peak-density	0.000	0.620	0.000	0.408	0.160	0.196	0.000	0.000
volume-density	0.000	0.548	**0.211**	0.291	0.172	0.171	0.000	0.000
mature-mean	0.000	0.498	0.000	0.428	0.161	0.210	0.000	0.366
no-penalty	0.000	0.566	0.000	0.445	0.176	0.211	**0.869**	**0.849**
norm-typicality	0.000	0.540	0.077	0.343	0.193	**0.249**	0.718	**0.821**
local-weights	0.000	0.526	**0.177**	0.431	0.181	**0.248**	0.806	**0.824**
density-sum	0.000	0.000	0.031	0.535	0.176	0.211	0.032	0.000
density-max	0.000	0.536	0.000	0.492	0.176	0.200	0.000	0.000
nearest	0.000	0.680	**0.180**	**0.593**	0.185	0.201	**0.897**	**0.848**

4.3.2 Combined Corrections

With *nearest* as the new control, Table 4.6 evaluates whether a second correction improves it. None does. Adding *peak-density* repeats the loss in DS3. Adding *no-penalty*, which improved the *vectorized* model on its own, lowers the Overall ARI of SCL-32. The combinations of three and four corrections inherit these losses.

Table 4.7 shows what *no-penalty* loses in SCL-32, through the partitions that the two models predict. It reports their homogeneity, which is one when every predicted cluster contains samples of a single true class, and their completeness, which is one when every true class is contained in a single predicted cluster [42]. The stream has three clusters and noise, which counts as a fourth class. With the penalty, the model predicts more clusters than that, with a high homogeneity: part of the noise forms macroclusters of its own. Without the penalty it predicts about three clusters and its homogeneity falls, because the noise is absorbed by the clusters. The ARI rewards the first behavior, but the benefit is accidental, since the penalty is not a treatment of

Table 4.6: Combined corrections: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
nearest	0.037	0.578	0.498	0.599	0.239	0.242	0.964	0.856
nearest + peak-density	0.061	0.603	0.468	0.605	0.220	0.270	0.897	0.857
nearest + peak-density + mature-mean	0.093	0.609	0.399	0.512	0.222	0.254	0.875	0.857
nearest + no-penalty	0.041	0.611	0.504	0.589	0.238	0.241	0.859	0.859
nearest + peak-density + no-penalty	0.055	0.597	0.430	0.605	0.225	0.267	0.860	0.859
nearest + peak-density + mature-mean + no-penalty	0.111	0.611	0.413	0.512	0.219	0.255	0.860	0.859
Steady-State ARI								
nearest	0.000	0.680	0.180	0.593	0.185	0.201	0.897	0.848
nearest + peak-density	0.010	0.827	0.188	0.623	0.166	0.195	0.881	0.833
nearest + peak-density + mature-mean	0.000	0.756	0.017	0.499	0.165	0.191	0.865	0.841
nearest + no-penalty	0.000	0.736	0.162	0.588	0.185	0.200	0.867	0.849
nearest + peak-density + no-penalty	0.010	0.750	0.188	0.623	0.165	0.191	0.867	0.849
nearest + peak-density + mature-mean + no-penalty	0.000	0.789	0.019	0.488	0.153	0.188	0.867	0.849

Table 4.7: Combined corrections: predicted partitions, medians over the development runs of the means over the batches.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Predicted clusters								
true clusters	3.0	3.0	4.1	3.0	9.9	8.8	4.0	4.0
nearest	1.6	2.4	5.2	3.6	23.7	19.8	4.5	3.9
nearest + no-penalty	1.7	2.7	4.9	3.8	23.6	19.3	2.9	2.9
Homogeneity								
nearest	0.31	0.64	0.51	0.74	0.77	0.68	0.92	0.76
nearest + no-penalty	0.34	0.70	0.50	0.74	0.77	0.68	0.76	0.76
Completeness								
nearest	0.90	0.95	0.70	0.74	0.56	0.55	0.90	0.90
nearest + no-penalty	0.91	0.92	0.70	0.72	0.56	0.55	0.92	0.92

noise.

4.3.3 The Corrected Model

The reference implementation assigns a sample x to the macrocluster that maximizes $\sum_l w^l \left(1 - \zeta^l(x)\right)$ over its active microclusters, see Section 3.1, where w^l is the share of the microcluster in the density of the macrocluster. For a microcluster with S samples, mean μ and variance σ^2,

$$1 - \zeta(x) = 1 - \frac{1}{2S}\left(1 + \frac{\|x - \mu\|^2}{\sigma^2}\right) \tag{4.1}$$

The standardized squared distance is divided by the number of samples, so the

more samples a microcluster has, the less the distance to it matters. A sample five standard deviations away from a microcluster with 500 samples scores 0.974 in Equation (4.1), more than the 0.950 scored by a sample one standard deviation away from a microcluster with 20 samples. Replacing $1 - \zeta$ by the normalized typicality of the formulation, $t = (1 - \xi)/(S - 2)$, reverses the bias instead of removing it: the values scale with $1/S$, and the microclusters with few samples dominate. The quantity that does not depend on the number of samples is the standardized squared distance itself,

$$q(x) = S\,\xi(x) - 1 = \frac{\|x - \mu\|^2}{\sigma^2} \tag{4.2}$$

which the outlier condition also thresholds, since Equation (2.5) is equivalent to $q(x) > m^2$. The membership *nearest* assigns a sample to the macrocluster of the active microcluster it is closest to in standard deviations.

The corrected model is therefore the *vectorized* model with the *nearest* membership, and it is the control of the experiments that follow. The order-dependent density and the dimensionality penalty remain in it: the experiments do not support replacing the former, and removing the latter costs the accidental separation of the noise.

4.4 Improvements

The candidates of this section modify the algorithm beyond what its formulation states. Most of them answer to what the corrected model still does wrong, which Table 4.8 summarizes through the partitions it predicts.

Table 4.8: Partitions predicted by the *corrected* model: medians over the development runs of the means over the batches. The noise label counts as one cluster, and the first batch, predicted before any sample is learned, is labeled as noise in full, the only noise of a model without a noise label.

Dataset	True clusters	Predicted clusters	Homogeneity	Completeness	True noise [%]	Predicted noise [%]
B1	3.0	1.6	0.31	0.90	9.0	3.1
B2	3.0	2.4	0.64	0.95	0.0	3.1
RBF	4.1	5.2	0.51	0.70	23.2	1.2
CSN	3.0	3.6	0.74	0.74	0.0	3.1
DS3	9.9	23.7	0.77	0.56	7.9	1.2
DS4	8.8	19.8	0.68	0.55	4.0	4.7
S32	4.0	4.5	0.92	0.90	9.0	2.9
S64	4.0	3.9	0.76	0.90	9.1	2.9

Three patterns stand out. In the static shapes of DS3 and DS4, the model predicts more than twice the true number of clusters, with high homogeneity and low completeness: the clusters it finds are pure, but each true cluster is split into several. In

StreamBenchmark(1), it predicts fewer clusters than there are, with low homogeneity: the two moving clusters are merged. Finally, the model never recognizes noise: in the six datasets that contain it, every noise sample is assigned to a cluster, and the predicted noise of the table comes from the first batch alone, predicted before any sample is learned.

A second limitation concerns the hyperparameter search. Table 4.9 compares the corrected model with *wide-range*, the same model with the upper bound of the search range of the variance limit extended from 10^2 to 10^4.

Table 4.9: Search range of the variance limit: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
corrected	0.037	0.578	0.498	0.599	0.239	0.242	0.964	0.856
wide-range	0.071	0.617	0.479	0.554	**0.376**	**0.357**	0.958	0.861
Steady-State ARI								
corrected	0.000	0.680	0.180	0.593	0.185	0.201	0.897	0.848
wide-range	0.010	0.810	0.086	0.543	**0.361**	0.000	0.897	0.835

Table 4.10: Search range of the variance limit: limit selected, median over the development runs, and number of the 32 runs whose penultimate batch holds a single predicted cluster.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Variance limit selected								
corrected	0.0259	0.0176	0.000993	0.0824	68.9	35.5	2.54	0.0117
wide-range	0.00643	0.00527	0.000785	0.0909	2.4e+03	1.77e+03	3.4	0.381
Runs that end with a single predicted cluster								
corrected	24	12	8	5	0	0	0	0
wide-range	16	7	8	8	5	19	0	0

By the decision rule the wider range improves the corrected model, in DS3 and DS4, whose selected limits are far above the original bound and orders of magnitude above those of the other two-dimensional datasets (Table 4.10): the variance limit depends on the scale of the data. We do not adopt the wider range. It changes the search and not the model, no fixed range suits every scale, and with limits of that size more than half of the runs of DS4 end with a single predicted cluster. One of the candidates of this section removes the dependence on the scale in the model instead.

Table 4.11 lists the candidates. They change the formation of the microclusters, the pruning, the formation of the macroclusters and the prediction, and three of them

reparameterize the sensitivity function.

Table 4.11: Candidate improvements, each a change of the corrected model.

Variant	Change
variance-cap	any sample that would raise the variance of a microcluster above r_0 is an outlier to it, whatever its size
relative-limit	r_0 a fraction of the variance of the stream, estimated recursively, searched in $[10^{-6}, 1]$
single-assign	only the accepting microcluster with the nearest mean absorbs a sample
forgetting	sample count of the recursive mean and variance capped at $S_{\max} \in [10, 10^4]$
chi-threshold	outlier condition $q > 1 + m\sqrt{2/d}$ instead of $q > m^2$, without the dimensionality penalty
recency	pruning rule of Galindo, Equation (2.15), for every microcluster
recency-inactive	the same rule for the inactive microclusters
soft-activation	a microcluster is active if its density reaches $f \in [0, 1.5]$ times the mean density of its component
no-activation	every microcluster with three samples or more is active
single-graph	macroclusters as the components of $\mathbb{G}_0$ with active microclusters, instead of those of $\mathbb{G}_1$
link-factor	factor $\ell \in [0.5, 4]$ instead of 2 in Equation (2.8), as Cardoso proposes in Equation (2.16) for the radii
noise-label	a sample whose distance q to every active microcluster exceeds n_σ^2, $n_\sigma \in [1, 10]$, is noise
m-reparam	function m of Equation (2.4) parameterized by its maximum and by the count at which it reaches 95% of it
m-constant	a constant $m \in [1, 5]$
m-asymptote	the asymptote of the original m searched in $[1.05, 10]$

4.4.1 Improvements One at a Time

Table 4.12 compares each candidate with the corrected model. Two candidates improve it. *single-assign* gains in the Overall ARI of StreamBenchmark(2), SCL-32 and SCL-64 and in the Steady-State ARI of StreamBenchmark(1), StreamBenchmark(2), SCL-32 and SCL-64, and *variance-cap* gains in the Steady-State ARI of RandomRBFGeneratorEvents.

Two other candidates produce the largest gains of the table together with a loss, so the rule classifies them as mixed. *noise-label* raises the Overall ARI of StreamBenchmark(1) and that of SCL-64, and loses in DS3. *relative-limit* gains in StreamBenchmark(1), StreamBenchmark(2) and DS3, and loses in the Steady-State ARI of DS4.

Table 4.12: Improvements, one at a time: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
corrected	0.037	0.578	0.498	0.599	0.239	0.242	0.964	0.856
variance-cap	0.043	0.617	0.522	0.502	0.234	0.273	0.867	0.859
soft-activation	0.028	0.567	0.444	0.573	0.215	0.240	0.978	0.855
no-activation	0.004	0.475	0.171	0.422	0.215	0.259	0.978	0.857
single-graph	0.020	0.589	0.434	0.680	0.236	0.253	0.969	0.857
link-factor	0.113	0.598	0.438	0.449	0.220	**0.271**	0.957	0.893
recency	0.052	0.555	0.388	0.497	0.172	**0.522**	0.871	0.856
recency-inactive	0.067	0.592	0.508	0.481	0.237	0.232	0.940	0.856
forgetting	0.048	0.601	0.494	0.560	0.238	0.230	0.954	0.857
noise-label	**0.919**	0.613	0.520	0.540	0.151	0.191	**0.982**	**1.000**
single-assign	0.151	**0.648**	0.512	0.465	0.234	0.246	**0.990**	**0.993**
chi-threshold	0.000	0.000	0.000	0.000	0.000	0.000	0.860	0.859
relative-limit	**0.085**	0.628	0.501	0.595	**0.351**	0.336	0.885	0.856
Steady-State ARI								
corrected	0.000	0.680	0.180	0.593	0.185	0.201	0.897	0.848
variance-cap	0.000	0.929	**0.504**	0.429	0.179	0.208	0.853	0.849
soft-activation	0.000	0.538	0.018	0.617	0.163	0.195	0.964	0.857
no-activation	0.000	0.000	0.015	0.100	0.142	0.203	0.972	0.847
single-graph	0.000	0.637	0.227	0.731	0.184	0.205	0.895	0.855
link-factor	0.025	0.806	0.226	0.321	0.157	0.209	0.900	0.850
recency	0.000	0.430	0.002	0.003	0.185	**0.482**	0.889	0.847
recency-inactive	0.000	0.675	0.140	0.390	0.185	0.185	0.904	0.840
forgetting	0.000	0.773	0.448	0.504	0.185	0.200	0.897	0.853
noise-label	**0.296**	0.855	0.363	0.515	0.163	0.167	**0.982**	**1.000**
single-assign	**0.000**	**0.938**	0.425	0.524	0.176	0.197	**0.983**	**0.990**
chi-threshold	0.000	0.000	0.000	0.000	0.000	0.000	0.867	0.849
relative-limit	**0.010**	**0.909**	0.254	0.573	**0.347**	0.023	0.855	0.835

Table 4.13: Hyperparameters selected for *noise-label* and share of the samples it labels as noise: medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Variance limit selected, searched up to 100								
corrected	0.0259	0.0176	0.000993	0.0824	68.9	35.5	2.54	0.0117
noise-label	0.000376	0.0262	0.00118	0.0765	95.3	94.8	2.28	0.0108
Noise distance selected, searched up to 10								
noise-label	7.1	3.5	5.7	6.8	9.1	9.3	4.8	3.8
Samples labeled as noise [%]								
true noise	9.0	0.0	23.2	0.0	7.9	4.0	9.0	9.1
noise-label	11.0	9.7	8.4	3.1	20.4	19.2	3.8	11.7

Table 4.14: Improvements, one at a time: predictions of a single cluster. Median share of the batches after the first, and number of the 32 development runs whose penultimate batch holds a single cluster.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Batches with a single predicted cluster [%]								
corrected	44	0	0	0	0	5	0	0
relative-limit	26	0	0	0	0	2	0	0
chi-threshold	100	100	99	100	100	100	0	0
Runs that end with a single predicted cluster								
corrected	24	12	8	5	0	0	0	0
relative-limit	16	3	9	5	4	10	0	0
chi-threshold	32	29	30	32	32	32	0	0

Both losses have identifiable causes. Table 4.13 shows that, in the two Chameleon datasets, the search selects for *noise-label* a variance limit and a noise distance at the upper end of their ranges, and that the model still labels as noise more than twice the share of true noise. Its microclusters are too small for the scale of these datasets, the limitation of the search range seen above, which *relative-limit* removes. Table 4.14 shows that *relative-limit* ends about a third of the runs of DS4 with a single predicted cluster, as *wide-range* does: without a bound, a variance limit that suits the first samples lets the microclusters grow until they merge the dataset, the growth that *variance-cap* prevents. The four changes are therefore evaluated in combination in Section 4.4.2.

The remaining candidates do not help. *forgetting* and *single-graph* are indistinguishable from the control. *recency* gains in DS4 and loses in StreamBenchmark(2) and Cassini, and *link-factor* gains in DS4 and loses in Cassini. *soft-activation*, *no-activation* and *recency-inactive* regress. Every relaxation of the activation criterion is thus neutral or harmful: the two-pass construction of the macroclusters, which Section 3.1 describes as a deviation of the reference implementation, is not a defect to be removed. *chi-threshold* collapses in every two-dimensional dataset, where Table 4.14 shows that it predicts a single cluster in practically every batch. For $d = 2$ its limit is $q \leq 1 + m$, which for the values of m close to one that Equation (2.4) gives to young microclusters is twice as permissive as $q \leq m^2$, and the microclusters grow until they form a single macrocluster.

4.4.2 Combined Improvements

We evaluate the four changes that helped in isolation in every combination of two, three and four of them. In the names of the combinations, *single*, *cap*, *noise* and *relative*

stand for *single-assign*, *variance-cap*, *noise-label* and *relative-limit*.

Table 4.15: Combined improvements: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
corrected	0.037	0.578	0.498	0.599	0.239	0.242	0.964	0.856
single + cap	**0.175**	0.637	**0.570**	0.475	0.239	0.222	**0.989**	**0.992**
single + noise	**0.621**	0.581	0.548	0.548	0.168	0.198	**0.990**	**0.992**
single + relative	**0.786**	0.625	**0.552**	0.554	**0.295**	0.246	**0.988**	**0.992**
cap + noise	**0.911**	0.595	0.536	0.556	0.153	0.198	**0.970**	0.995
cap + relative	**0.748**	0.621	**0.534**	0.577	**0.272**	**0.380**	0.941	0.857
noise + relative	**0.808**	**0.646**	0.492	0.581	**0.309**	0.334	**0.995**	**1.000**
single + cap + noise	**0.629**	0.621	0.530	0.514	0.160	0.177	**0.988**	**0.993**
single + cap + relative	**0.812**	0.621	**0.552**	0.478	0.231	0.272	**0.988**	**0.992**
single + noise + relative	**0.915**	**0.643**	0.510	0.455	**0.304**	0.255	**0.987**	**0.990**
cap + noise + relative	**0.775**	0.623	0.532	0.590	0.267	**0.386**	**0.982**	1.000
single + cap + noise + relative	**0.879**	0.649	**0.567**	0.515	0.236	0.271	**0.985**	**0.990**
Steady-State ARI								
corrected	0.000	0.680	0.180	0.593	0.185	0.201	0.897	0.848
single + cap	**0.000**	**0.867**	**0.478**	0.462	0.180	0.176	**0.980**	**0.988**
single + noise	**0.221**	**0.806**	**0.422**	0.576	0.181	0.150	**0.984**	**0.988**
single + relative	**0.710**	**0.867**	**0.473**	0.512	**0.341**	**0.328**	**0.982**	**0.986**
cap + noise	**0.290**	0.747	**0.553**	0.548	0.156	0.159	0.967	1.000
cap + relative	**0.271**	**0.896**	**0.514**	0.564	0.248	**0.351**	0.930	0.833
noise + relative	**0.383**	0.869	0.292	0.551	0.319	0.023	**1.000**	**1.000**
single + cap + noise	**0.215**	**0.878**	**0.489**	0.470	0.160	0.132	**0.984**	**0.990**
single + cap + relative	**0.834**	0.876	**0.444**	0.519	0.148	**0.232**	**0.982**	**0.990**
single + noise + relative	**0.865**	**0.859**	**0.395**	0.451	**0.338**	**0.320**	**0.983**	**0.982**
cap + noise + relative	**0.488**	0.744	**0.502**	0.556	**0.249**	**0.408**	0.992	1.000
single + cap + noise + relative	**0.752**	0.843	**0.397**	0.443	0.161	**0.307**	**0.984**	**0.986**

Table 4.15 shows that nine of the eleven combinations improve the corrected model, and the two that do not, *single + cap* and *cap + noise*, each lose in one dataset, Cassini and DS3 respectively. The two losses explained in the previous subsection disappear as predicted: *noise + relative* gains in DS3, where *noise-label* alone lost, and *cap + relative* gains in DS4, where *relative-limit* alone lost.

Since several combinations improve the control, we choose the improved model in two steps: no other combination may improve it in a direct comparison, and, among those that satisfy this, it must have the most significant gains over the corrected model. *single + relative* and *single + noise + relative* have the most, twelve each, and Table 4.16 gives the direct comparisons between the leading combinations.

single + noise + relative improves *single + relative*, in StreamBenchmark(1), and no combination improves it; its comparison with *noise + relative* is mixed, and adding *variance-cap* to it regresses in DS3. The only other combination that no other improves,

Table 4.16: Direct comparisons between combinations: clustering quality, medians over the development runs.

Model	Control	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI									
single + relative		0.786	0.625	0.552	0.554	0.295	0.246	0.988	0.992
noise + relative		0.808	0.646	0.492	0.581	0.309	0.334	0.995	1.000
single + noise + relative	*single + relative*	**0.915**	0.643	0.510	0.455	0.304	0.255	0.987	0.990
single + noise + relative	*noise + relative*	0.915	0.643	0.510	0.455	0.304	0.255	0.987	0.990
single + noise + relative	*cap + noise + relative*	**0.915**	0.643	0.510	0.455	0.304	0.255	0.987	0.990
cap + noise + relative	*noise + relative*	0.775	0.623	0.532	0.590	0.267	**0.386**	0.982	1.000
single + cap + noise + relative	*single + noise + relative*	0.879	0.649	0.567	0.515	0.236	0.271	0.985	0.990
single + cap + noise + relative	*cap + noise + relative*	0.879	0.649	0.567	0.515	0.236	0.271	0.985	0.990
Steady-State ARI									
single + relative		0.710	0.867	0.473	0.512	0.341	0.328	0.982	0.986
noise + relative		0.383	0.869	0.292	0.551	0.319	0.023	1.000	1.000
single + noise + relative	*single + relative*	**0.865**	0.859	0.395	0.451	0.338	0.320	0.983	0.982
single + noise + relative	*noise + relative*	**0.865**	0.859	0.395	0.451	0.338	**0.320**	0.983	0.982
single + noise + relative	*cap + noise + relative*	0.865	0.859	0.395	0.451	0.338	0.320	0.983	0.982
cap + noise + relative	*noise + relative*	0.488	0.744	0.502	0.556	0.249	**0.408**	0.992	1.000
single + cap + noise + relative	*single + noise + relative*	0.752	0.843	0.397	0.443	0.161	0.307	0.984	0.986
single + cap + noise + relative	*cap + noise + relative*	0.752	0.843	0.397	0.443	0.161	0.307	0.984	0.986

cap + noise + relative, has seven gains over the corrected model, and the two differ by a trade-off that the rule does not resolve: the first is better in StreamBenchmark(1), the second in DS4.

The improved model is therefore the corrected model with *single-assign*, *noise-label* and *relative-limit*. *single-assign* keeps overlapping microclusters from absorbing the same samples, *noise-label* lets the model recognize noise, which the corrected model never does (Table 4.8), and *relative-limit* frees the variance limit from the scale of the data (Table 4.10); Section 4.7 describes the resulting model in full. Compared with the corrected model, it gains in the Overall ARI of StreamBenchmark(1), StreamBenchmark(2), DS3, SCL-32 and SCL-64 and in the Steady-State ARI of every dataset except Cassini, where its medians are lower without the difference being significant.

4.4.3 Refinements of the Improved Model

We evaluate four further changes on the improved model. *forgetting* and *recency* did not help the corrected model, but they target the tracking of moving clusters, which the improved model now does better. The removal of the dimensionality penalty improved the *vectorized* model, but we rejected it in Section 4.3 because the penalty accidentally separated part of the noise; we reconsider it now that the model labels the noise explicitly. *persistent-labels* makes each macrocluster keep the label of the macrocluster with which it shares most of its mass, instead of being renumbered at every update: the partition of a batch does not change, but the objective of the search computes the ARI over all batches together (Section 3.6.1), where a renumbering counts as a

disagreement.

Table 4.17: Refinements of the improved model: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
improved	0.915	0.643	0.510	0.455	0.304	0.255	0.987	0.990
improved + no-penalty	0.864	0.696	0.508	0.454	0.304	0.255	**1.000**	**1.000**
improved + persistent-labels	0.887	0.657	0.524	0.536	0.311	0.283	0.990	0.991
improved + forgetting	0.903	0.640	0.550	0.593	0.291	0.264	0.981	0.990
improved + recency	0.890	0.582	0.506	0.476	0.218	0.213	0.988	0.994
Steady-State ARI								
improved	0.865	0.859	0.395	0.451	0.338	0.320	0.983	0.982
improved + no-penalty	0.783	0.913	0.395	0.461	0.338	0.320	**1.000**	**1.000**
improved + persistent-labels	0.833	0.925	0.461	0.540	0.305	0.328	0.985	0.982
improved + forgetting	0.915	0.854	0.498	0.601	0.307	0.289	0.950	0.986
improved + recency	0.732	0.803	0.336	0.457	0.221	0.328	0.974	0.990

Only the removal of the penalty improves the improved model in Table 4.17: it takes the Overall and Steady-State ARI of SCL-32 and SCL-64 to 1.000. *persistent-labels* and *forgetting* are indistinguishable from the control, and *recency* regresses in DS3 and DS4.

The first two rows of each panel also give a measure of the noise of the protocol. In the six two-dimensional datasets the factor $2/d$ is one, so *improved* and *improved + no-penalty* are the same algorithm, tuned twice with the same seeds. Their medians still differ, most of all in StreamBenchmark(1) and StreamBenchmark(2), and none of these differences is significant.

Table 4.18: Two hyperparameter searches of a single algorithm with the same seeds, *improved* and *improved + no-penalty*, in the two-dimensional datasets: number of the 32 development runs.

	B1	B2	RBF	CSN	DS3	DS4
Runs						
both searches select the same hyperparameters	5	11	31	24	32	27
both evaluations have the same Overall ARI	6	11	31	25	32	28

Table 4.18 locates the origin of these differences in the search. The two searches select the same hyperparameters in most of the runs of four datasets and in a minority of the runs of StreamBenchmark(1) and StreamBenchmark(2), and wherever they do, the two evaluations coincide. The sampler of the search is seeded and the algorithm is deterministic, so the execution time that the objective measures is the only input

that differs between the two searches. Differences of this size between distinct models should therefore not be read as effects.

The model that results, MicroTEDAClus+, is therefore the improved model without the dimensionality penalty.

4.4.4 Variants Not Retained

Three further families of variants were evaluated by the same rule, and none improves its control (Table 4.19). Against the corrected model, the reparameterizations of the sensitivity function each add one or two hyperparameters to a search with a fixed budget of trials: *m-reparam* and *m-asymptote* regress, and *m-constant* gains in StreamBenchmark(1) and StreamBenchmark(2) at the cost of five of the remaining six datasets. Against MicroTEDAClus+, a covariance matrix replaces the scalar variance of a microcluster, with the Mahalanobis distance of Section 2.1 and the intersection condition generalized along the line that joins the two means, *covariance, directional link*, or by the Bhattacharyya distance, *covariance, Bhattacharyya link*; both regress, in StreamBenchmark(2), and neither gains, not even in the elongated static shapes. The two order-independent densities of Section 4.3, tested again on MicroTEDAClus+, where the density only decides which microclusters are active, regress too: *peak-density* in DS3 and *volume-density* in five datasets. We retain the original function m, the scalar variance and the density of the last absorbed sample.

Table 4.19: Variants not retained: clustering quality, medians over the development runs. Each variant is marked against the control of its row.

Model	Control	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI									
corrected		0.037	0.578	0.498	0.599	0.239	0.242	0.964	0.856
MicroTEDAClus+		0.864	0.696	0.508	0.454	0.304	0.255	1.000	1.000
m-reparam	*corrected*	0.009	0.544	0.407	0.581	0.234	0.246	0.857	0.853
m-constant	*corrected*	**0.224**	0.613	0.431	0.282	0.000	0.203	0.855	0.853
m-asymptote	*corrected*	0.048	0.561	0.445	0.450	0.196	0.230	0.859	0.858
covariance, directional link	MicroTEDAClus+	0.897	0.607	0.524	0.467	0.303	0.239	1.000	1.000
covariance, Bhattacharyya link	MicroTEDAClus+	0.899	0.628	0.551	0.396	0.310	0.227	1.000	1.000
peak-density	MicroTEDAClus+	0.852	0.625	0.508	0.577	0.208	0.359	1.000	1.000
volume-density	MicroTEDAClus+	0.589	0.631	0.435	0.312	0.156	0.229	1.000	1.000
Steady-State ARI									
corrected		0.000	0.680	0.180	0.593	0.185	0.201	0.897	0.848
MicroTEDAClus+		0.783	0.913	0.395	0.461	0.338	0.320	1.000	1.000
m-reparam	*corrected*	0.000	0.471	0.235	0.502	0.182	0.194	0.831	0.821
m-constant	*corrected*	**0.110**	**0.892**	0.487	0.326	0.000	0.147	0.856	0.823
m-asymptote	*corrected*	0.010	0.499	0.307	0.421	0.113	0.218	0.847	0.831
covariance, directional link	MicroTEDAClus+	0.881	0.748	0.434	0.479	0.331	0.233	1.000	1.000
covariance, Bhattacharyya link	MicroTEDAClus+	0.865	0.863	0.404	0.381	0.335	0.245	1.000	1.000
peak-density	MicroTEDAClus+	0.742	0.828	0.347	0.558	0.000	0.345	1.000	1.000
volume-density	MicroTEDAClus+	0.592	0.716	0.333	0.326	0.120	0.195	1.000	1.000

4.5 Validation

We tuned and evaluated the four models of the chain, *vectorized*, *corrected*, *improved* and MicroTEDAClus+, again on the 94 validation runs, which took no part in any of the choices above, as we set in Section 4.1.3. We executed the experiments on the covariance and on the order-independent densities of Section 4.4.4 after these runs. Both rejected their candidates, so we changed no model after we saw the validation runs.

4.5.1 Clustering Quality

Table 4.20 compares each model with its predecessor, and Table 4.21 compares MicroTEDAClus+ with the *vectorized* model directly.

Table 4.20: Clustering quality of the accepted models: median and quartiles over the validation runs. Each model is compared with the model to its left.

Dataset	*vectorized*	*corrected*	*improved*	MicroTEDAClus+
Overall ARI				
B1	0.000 (0.000, 0.348)	0.059 (0.000, 0.517)	**0.843 (0.635, 0.934)**	0.853 (0.704, 0.937)
B2	0.573 (0.512, 0.634)	**0.607 (0.534, 0.677)**	**0.645 (0.587, 0.704)**	0.627 (0.576, 0.704)
RBF	0.421 (0.000, 0.547)	0.364 (0.064, 0.560)	**0.488 (0.323, 0.627)**	0.488 (0.338, 0.627)
CSN	0.490 (0.352, 0.562)	0.494 (0.329, 0.627)	0.497 (0.406, 0.634)	0.481 (0.396, 0.618)
DS3	0.231 (0.221, 0.232)	**0.238 (0.230, 0.241)**	**0.290 (0.266, 0.322)**	0.290 (0.268, 0.322)
DS4	0.248 (0.218, 0.289)	0.247 (0.223, 0.269)	**0.308 (0.240, 0.355)**	0.308 (0.240, 0.354)
S32	0.869 (0.650, 0.923)	**0.910 (0.855, 0.974)**	**0.988 (0.977, 0.992)**	**1.000 (1.000, 1.000)**
S64	0.675 (0.000, 0.872)	**0.859 (0.849, 0.953)**	**0.993 (0.952, 1.000)**	1.000 (1.000, 1.000)
Steady-State ARI				
B1	0.000 (0.000, 0.000)	0.000 (0.000, 0.056)	**0.712 (0.184, 0.919)**	0.782 (0.195, 0.963)
B2	0.541 (0.000, 0.833)	**0.802 (0.464, 0.921)**	0.842 (0.657, 0.966)	0.853 (0.692, 0.938)
RBF	0.003 (0.000, 0.530)	0.054 (0.000, 0.459)	**0.474 (0.182, 0.663)**	0.481 (0.211, 0.663)
CSN	0.374 (0.113, 0.527)	0.433 (0.249, 0.645)	0.483 (0.378, 0.660)	0.483 (0.349, 0.656)
DS3	0.176 (0.173, 0.184)	**0.184 (0.179, 0.187)**	**0.295 (0.232, 0.364)**	0.296 (0.234, 0.365)
DS4	0.210 (0.176, 0.224)	0.199 (0.175, 0.212)	**0.323 (0.210, 0.358)**	0.322 (0.217, 0.349)
S32	0.140 (0.000, 0.908)	**0.873 (0.812, 0.969)**	**0.975 (0.933, 0.994)**	**1.000 (1.000, 1.000)**
S64	0.000 (0.000, 0.000)	**0.869 (0.818, 0.908)**	**0.990 (0.925, 0.998)**	1.000 (1.000, 1.000)

Every step of the chain is confirmed: each model improves its predecessor and none regresses on the two metrics of the decision rule. The corrected model gains in StreamBenchmark(2), DS3 and the two scalability streams, the improved model in most datasets, and MicroTEDAClus+ in SCL-32.

From the *vectorized* model to MicroTEDAClus+, the Overall ARI improves in every dataset except Cassini and the Steady-State ARI improves in all eight. The largest changes are in the streams with noise and movement. The gains in the static shapes are significant but modest: in Cassini, DS3 and DS4 MicroTEDAClus+ stays below

Table 4.21: MicroTEDAClus+ against the *vectorized* model: medians over the validation runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
vectorized	0.000	0.573	0.421	0.490	0.231	0.248	0.869	0.675
MicroTEDAClus+	**0.853**	**0.627**	**0.488**	0.481	**0.290**	**0.308**	**1.000**	**1.000**
Steady-State ARI								
vectorized	0.000	0.541	0.003	0.374	0.176	0.210	0.140	0.000
MicroTEDAClus+	**0.782**	**0.853**	**0.481**	**0.483**	**0.296**	**0.322**	**1.000**	**1.000**

0.5.

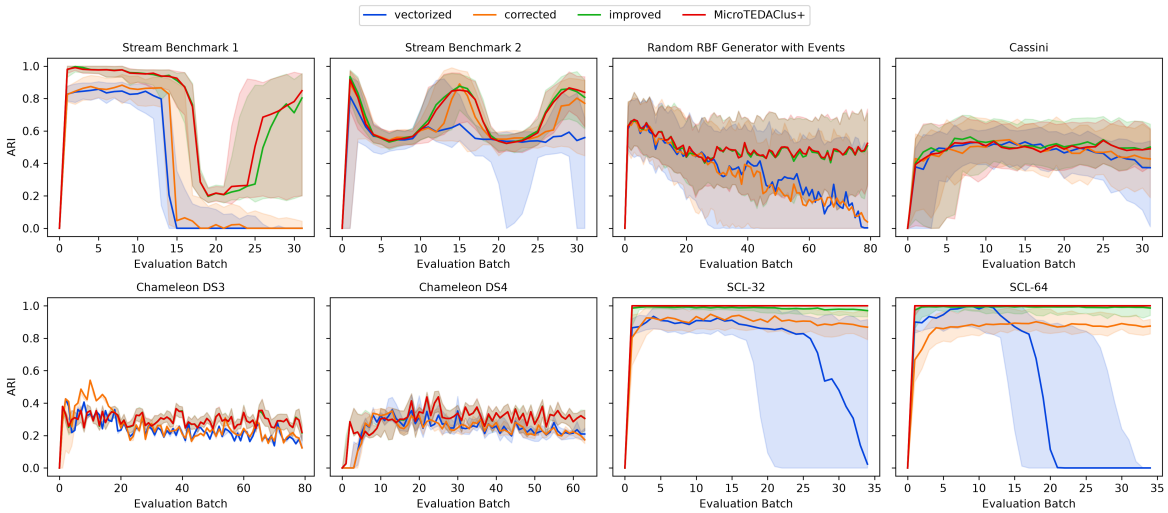


Figure 4.1: ARI of each evaluation batch for the accepted models: median and interquartile range over the 94 validation runs.

Figure 4.1 shows where the gains come from. In StreamBenchmark(1), the *vectorized* and corrected models lose the two clusters when they cross and do not recover, while the improved model and MicroTEDAClus+ separate them again. In RandomRBFGeneratorEvents, the first two models degrade along the stream and the last two hold their level. In SCL-32 and SCL-64, the *vectorized* model degrades, the corrected model stabilizes below one because it assigns the noise to the clusters, and the models with the noise label reach one.

4.5.2 Cost

We measure the execution times of the validation runs, like those of every experiment of this chapter, under parallel load. Table 4.22 gives the cost of the accepted models and of the reference implementation measured by the procedure of Section 4.2.2: each

model replays the evaluation segment of each of the 32 development runs with the hyperparameters that were tuned for it in that run. The times therefore include the effect of the hyperparameters that each model leads the search to select, which is the cost of the model as it is used.

Table 4.22: Cost of the original implementation and of the accepted models on the evaluation segment, each with its own tuned hyperparameters: median and quartiles over the 32 development runs. Each task runs alone, and its time is the smallest of three executions, which differ by 0.8% in the median.

Dataset	*original*	*vectorized*	*corrected*	*improved*	MicroTEDAClus+
Learning time [ms]					
B1	552 (271, 3404)	38.0 (27.6, 65.3)	29.1 (25.6, 66.3)	144 (56.4, 233)	152 (74.4, 221)
B2	981 (160, 1426)	36.8 (21.9, 41.0)	34.6 (22.8, 40.2)	34.3 (30.8, 38.7)	32.5 (29.5, 37.1)
RBF	16002 (4426, 32407)	243 (92.0, 502)	420 (133, 679)	441 (176, 797)	500 (182, 798)
CSN	1107 (910, 1383)	37.5 (34.9, 40.8)	38.9 (37.1, 52.7)	36.8 (32.5, 45.9)	36.9 (32.2, 49.0)
DS3	8876 (8711, 21640)	149 (146, 160)	146 (144, 161)	91.1 (88.9, 91.6)	90.7 (88.7, 91.6)
DS4	9154 (7762, 15594)	153 (131, 216)	148 (128, 152)	73.0 (70.0, 428)	73.2 (69.6, 425)
S32	665 (359, 1723)	35.3 (28.7, 56.7)	33.6 (27.6, 77.4)	31.5 (30.0, 33.0)	30.6 (29.2, 43.4)
S64	701 (423, 1679)	41.1 (34.5, 78.7)	66.5 (39.1, 152)	33.5 (30.4, 36.5)	32.1 (30.8, 37.8)
Prediction time [ms]					
B1	61.2 (38.2, 533)	5.2 (4.6, 6.3)	5.0 (4.8, 6.5)	9.3 (7.4, 10.8)	9.4 (8.2, 10.6)
B2	97.0 (31.7, 134)	5.2 (4.5, 5.6)	5.4 (4.7, 5.7)	6.1 (5.9, 6.5)	6.0 (5.8, 6.3)
RBF	1699 (472, 4993)	16.8 (13.7, 25.1)	25.6 (16.5, 31.1)	28.4 (20.5, 37.5)	30.0 (21.0, 37.5)
CSN	115 (98.0, 148)	5.3 (5.2, 5.4)	5.9 (5.7, 6.1)	6.5 (6.0, 6.9)	6.4 (6.0, 7.0)
DS3	807 (745, 1069)	16.6 (16.4, 17.0)	17.5 (17.1, 17.7)	16.2 (16.0, 16.3)	16.1 (16.0, 16.3)
DS4	620 (409, 1092)	12.4 (11.8, 13.9)	12.9 (12.2, 14.2)	13.1 (12.8, 27.4)	13.1 (12.8, 27.4)
S32	105 (76.9, 339)	6.3 (5.9, 8.3)	6.4 (6.1, 7.3)	7.8 (7.4, 8.1)	6.4 (6.4, 6.8)
S64	114 (86.5, 176)	7.1 (6.5, 8.3)	7.9 (6.9, 9.7)	8.1 (7.5, 8.7)	6.7 (6.6, 6.7)
Total time relative to the vectorized model, per run					
B1	13.56 (8.32, 48.93)	1.00 (1.00, 1.00)	0.94 (0.70, 1.08)	2.02 (1.24, 4.80)	2.67 (1.33, 5.11)
B2	25.43 (6.91, 32.56)	1.00 (1.00, 1.00)	0.98 (0.69, 1.43)	1.04 (0.87, 1.34)	0.95 (0.83, 1.34)
RBF	57.53 (32.46, 75.57)	1.00 (1.00, 1.00)	1.19 (0.85, 2.05)	1.42 (0.59, 6.31)	1.41 (0.60, 6.34)
CSN	27.72 (23.82, 38.97)	1.00 (1.00, 1.00)	1.04 (0.98, 1.45)	1.00 (0.92, 1.23)	1.00 (0.91, 1.25)
DS3	59.40 (56.86, 131.65)	1.00 (1.00, 1.00)	0.99 (0.94, 1.04)	0.63 (0.59, 0.66)	0.63 (0.59, 0.66)
DS4	59.68 (50.93, 101.93)	1.00 (1.00, 1.00)	0.98 (0.77, 1.03)	0.62 (0.53, 2.22)	0.62 (0.51, 2.22)
S32	18.09 (12.55, 26.95)	1.00 (1.00, 1.00)	0.99 (0.72, 1.75)	0.96 (0.60, 1.09)	0.94 (0.62, 1.26)
S64	16.63 (11.12, 32.53)	1.00 (1.00, 1.00)	1.47 (1.00, 1.86)	0.88 (0.46, 1.00)	0.81 (0.53, 1.03)
Steady-state microcluster count					
B1	8 (3, 94)	14 (4, 66)	8 (4, 64)	347 (86, 588)	366 (132, 600)
B2	20 (5, 46)	28 (6, 51)	19 (5, 39)	26 (18, 42)	23 (15, 32)
RBF	92 (11, 304)	104 (7, 278)	197 (26, 464)	374 (104, 688)	380 (130, 697)
CSN	36 (22, 53)	36 (30, 42)	45 (35, 60)	36 (23, 52)	36 (23, 58)
DS3	118 (114, 235)	115 (112, 118)	115 (113, 117)	26 (25, 28)	26 (25, 28)
DS4	129 (114, 225)	124 (111, 222)	120 (111, 125)	31 (29, 346)	31 (29, 346)
S32	6 (5, 24)	6 (5, 28)	6 (5, 100)	13 (11, 15)	12 (10, 22)
S64	6 (4, 20)	9 (4, 40)	28 (14, 170)	10 (7, 13)	11 (7, 14)

The corrected model costs the same as the *vectorized* one: the median of the ratio between their total times, computed per run, is near one in seven datasets. The exception is SCL-64, where the corrected model keeps more microclusters than the *vectorized* model. MicroTEDAClus+ takes about six tenths of the time of the *vectorized*

model in DS3 and DS4, almost the same in StreamBenchmark(2), Cassini, SCL-32 and SCL-64, and more in RandomRBFGeneratorEvents and in StreamBenchmark(1).

These differences follow the number of microclusters that the tuned models keep, not the cost of the changes: MicroTEDAClus+ keeps tens of microclusters in DS3 and DS4, where the *vectorized* model keeps hundreds, and many more in StreamBenchmark(1), where it is slowest relative to that model but still learns the evaluation segment faster than the reference implementation. The removal of the dimensionality penalty costs nothing: the improved model and MicroTEDAClus+ have nearly the same times in SCL-32 and SCL-64, and in the two-dimensional datasets, where they are the same algorithm, they differ only as their searches do. The prediction time changes little along the chain.

The quartiles of Table 4.22 are wide not because of the measurement, whose three executions of a task agree closely, but because of the search, which in a quarter of the runs of DS4 selects configurations that keep hundreds of microclusters or more, against a median in the tens.

4.6 Limits of the Hyperparameter Selection

MicroTEDAClus+ stays below an Overall ARI of 0.5 in the three datasets of static shapes (Table 4.20). Since every result measures a model as the protocol tunes it, a low score can come from the model or from hyperparameters that the search does not select. To separate the two, we draw 400 configurations at random from the search space of MicroTEDAClus+, with the distributions that the search samples from, and score each of them twice: by the objective of the search on the first 20% of the stream (Algorithm 1), which is what the search sees, and by the evaluation metrics on the remaining 80%, which it is meant to maximize. We also probe an extended space that adds six design choices of this chapter as categorical hyperparameters. In Cassini, each configuration is scored on the streams of the first four development runs; the Chameleon datasets have a single stream.

The search space holds configurations far better than those the protocol selects (Table 4.23): the best has an Overall ARI above one half in the three datasets. The objective ranks them low, with less than half of the objective of the configuration it prefers, whose evaluation is close to that of the protocol: the search finds what the objective prefers. The best configurations are fine-grained, with a lower variance limit, a decay in the thousands and far more microclusters, and the extended space holds better ones still. The figures of the Chameleon datasets are optimistic, since the best of 400 configurations is chosen on the stream on which it is scored. The modest scores

Table 4.23: Reachability probe: for each search space, the configuration of highest objective and the one of highest Overall ARI. The first row of each dataset is MicroTEDAClus+ as the protocol tunes it, with medians over the development runs; the microcluster count is that of the penultimate batch.

Dataset	Space	Configuration	Objective	Overall ARI	Steady-State ARI	Microclusters	r	decay
CSN	final	tuned by the protocol	-	0.454	0.460	36	0.0609	46
	MicroTEDAClus+	highest objective	0.436	0.467	0.426	32	0.0621	40
	MicroTEDAClus+	highest Overall ARI	0.150	0.883	0.917	418	0.0086	1944
	extended	highest objective	0.445	0.423	0.043	4	0.2391	45
	extended	highest Overall ARI	0.092	0.977	0.996	610	0.0064	3507
DS3	final	tuned by the protocol	-	0.304	0.337	26	0.0610	34
	MicroTEDAClus+	highest objective	0.395	0.275	0.267	30	0.0570	52
	MicroTEDAClus+	highest Overall ARI	0.057	0.587	0.625	1238	0.0032	8694
	extended	highest objective	0.261	0.302	0.265	47	0.1648	50
	extended	highest Overall ARI	0.049	0.721	0.797	1445	0.0024	3857
DS4	final	tuned by the protocol	-	0.254	0.320	31	0.0559	51
	MicroTEDAClus+	highest objective	0.335	0.281	0.341	27	0.0570	52
	MicroTEDAClus+	highest Overall ARI	0.043	0.514	0.474	1104	0.0032	8694
	extended	highest objective	0.274	0.352	0.352	23	0.0609	16
	extended	highest Overall ARI	0.113	0.594	0.657	426	0.0050	159

of MicroTEDAClus+ in the static shapes are thus, at least in part, a limitation of the selection of its hyperparameters and not of the algorithm, and every comparison of this chapter is conditional on the protocol, which places every model on equal terms.

4.7 MicroTEDAClus+

MicroTEDAClus+ keeps the structure of MicroTEDAClus: microclusters updated recursively with the TEDA statistics, macroclusters as connected components of the graph of intersecting active microclusters, and pruning of the inactive microclusters by vitality. It differs from the algorithm of the reference implementation in the five points that the experiments accepted, listed in Table 4.24. It has three hyperparameters: the relative variance limit r, the decay, with $\rho = 1/\text{decay}$, and the noise distance n_σ.

4.7.1 Microclusters

A microcluster i is described by its number of samples S^i, its mean μ^i, its variance $(\sigma^i)^2$, its density D^i, its vitality v^i and a flag that tells whether it is active. The model also keeps the variance of the stream, σ^2_{stream}, the mean squared distance of the samples seen so far to their mean, updated before each batch is learned. For a sample x and a microcluster i,

Table 4.24: Differences between MicroTEDAClus+ and the algorithm of the reference implementation of MicroTEDAClus.

	MicroTEDAClus	MicroTEDAClus+
Membership	macrocluster with the largest density-weighted sum of $1-\zeta$	macrocluster of the active microcluster with the smallest standardized distance
Update	every microcluster that accepts the sample	the nearest microcluster that accepts the sample
Noise	every sample is assigned to a macrocluster	a sample farther than n_σ standard deviations from every active microcluster is noise
Variance limit	absolute, r_0	relative to the variance of the stream, $r_0 = r\,\sigma^2_{\text{stream}}$
Distances	scaled by $2/d$	not scaled

$$S' = S^i + 1, \qquad \mu' = \frac{S'-1}{S'}\mu^i + \frac{x}{S'}, \qquad \sigma'^2 = \frac{S'-1}{S'}(\sigma^i)^2 + \frac{\|x-\mu'\|^2}{S'-1}, \qquad \xi' = \frac{1}{S'} + \frac{\|x-\mu'\|^2}{S'\sigma'^2} \tag{4.3}$$

where $\xi' = 1/S'$ when $\sigma'^2 = 0$. The sample is an outlier to the microcluster, outlier $\in$ [false, true], if

$$\text{outlier} = \begin{cases} \sigma'^2 > r\,\sigma^2_{\text{stream}}, & S' < 3 \\ \dfrac{\xi'}{2} > \dfrac{m(S')^2+1}{2S'}, & S' \geq 3 \end{cases} \tag{4.4}$$

Algorithm 2 processes a sample.

4.7.2 Macroclusters

The macroclusters are rebuilt after each batch. Two microclusters intersect when Equation (2.8) holds between their means and standard deviations, which defines the graph $\mathbb{G}_0$. In each connected component of $\mathbb{G}_0$, a microcluster is active if it has at least three samples and its density is not below the mean density of the microclusters of the component. The macroclusters are the connected components of $\mathbb{G}_1$, the subgraph of $\mathbb{G}_0$ induced by the active microclusters.

Algorithm 2 Microcluster update of MicroTEDAClus+ for a sample x

Require: x, the microclusters, r, ρ, σ^2_{stream}
$A \leftarrow$ the microclusters to which x is not an outlier by Equation (4.4)
if $A = \emptyset$ **then**
    create a microcluster with $S = 1$, $\mu = x$, $\sigma^2 = 0$, $D = 0$, $v = 1$, flagged as active
else
    $i \leftarrow \arg\min_{l \in A} \|x - \mu^l\|$
    $v^i \leftarrow v^i + \rho\,(\sigma^i - \|x - \mu^i\|)/\sigma^i$ if $\sigma^i > 0$, otherwise $v^i \leftarrow 1$
    $S^i \leftarrow S'$, $\mu^i \leftarrow \mu'$, $(\sigma^i)^2 \leftarrow \sigma'^2$, $D^i \leftarrow 2/\xi'$, from Equation (4.3)
end if
for each microcluster l that is not active **do**
    $v^l \leftarrow v^l - \rho$
    remove l if $v^l < 0$
end for

4.7.3 Prediction

For $q^l(x) = \|x - \mu^l\|^2/(\sigma^l)^2$, the standardized squared distance of a sample to an active microcluster l, and $l^* = \arg\min_l q^l(x)$,

$$l_x = \begin{cases} -1, & q^{l^*}(x) > n_\sigma^2 \\ l_{\text{macro}}(l^*), & \text{otherwise} \end{cases} \tag{4.5}$$

Before the first batch is learned there is no active microcluster, and every sample is labeled as noise.

4.7.4 What Remains of the Original Model

The function m, the density, the activation criterion, the two-pass construction of the macroclusters and the vitality mechanism are those of the reference implementation. We challenged each of them in this chapter with at least one alternative and none of the alternatives improved the model. In particular, the density of a microcluster is still the density of the last sample it absorbed, the order dependence identified in Section 2.2.1: in MicroTEDAClus+ it only decides which microclusters are active, and Section 4.4.4 verifies whether an order-independent density can replace it there.

We search the hyperparameters in $r \in [10^{-6}, 1]$ and decay $\in [10, 10^4]$, both on a logarithmic scale, and $n_\sigma \in [1, 10]$.

In the experiments of this chapter, we execute MicroTEDAClus+ as a configuration of the implementation of Section 4.2. For the main experiment, we implement the model on its own, with its three hyperparameters and without the options of the ex-

perimental implementation, and we verified that the two implementations produce the same predictions and the same number of microclusters, batch by batch, on scalability streams with 2, 8 and 32 dimensions.

4.8 MicroTEDAClus+ in the Main Experiment

The experiments above compare models that share one implementation, on runs reserved for the development of the model. In this section, we return to the main experiment of Chapter 3. We tune and evaluate MicroTEDAClus+ on its 94 runs, six datasets and seven scalability streams, by the same code and on the same machine as the benchmark models, and compare it with MicroTEDAClus and the six benchmark models as Section 3.8 compares MicroTEDAClus. The streams of these runs took no part in the development of the model, except those of DS3 and DS4, which are the same in every run (Section 4.1.3).

4.8.1 Clustering Quality

Table 4.25: Clustering quality of MicroTEDAClus+, MicroTEDAClus and the benchmark models: medians over the runs of the main experiment.

Model	B1	B2	RBF	CSN	DS3	DS4
Overall ARI						
MicroTEDAClus+	0.894	0.634	0.491	0.496	0.288	0.296
Better/same/worse	7/0/0	7/0/0	6/1/0	4/1/2	5/0/2	5/1/1
MicroTEDAClus	**0.035**	**0.564**	**0.359**	0.492	**0.215**	**0.202**
DenStream	**0.665**	**0.402**	**0.360**	**0.383**	**0.174**	**0.184**
CluStream	**0.096**	**0.566**	**0.170**	0.573	0.324	0.319
StreamKM	**0.183**	**0.498**	**0.371**	0.696	0.330	0.327
AutoCloud	**0.043**	**0.001**	**0.051**	**0.076**	**0.023**	**0.030**
CEDAS	**0.252**	**0.000**	**0.178**	**0.000**	**0.000**	**0.000**
TEDACloud	**0.827**	**0.509**	0.495	**0.360**	**0.040**	**0.125**
Steady-State ARI						
MicroTEDAClus+	0.869	0.842	0.495	0.500	0.300	0.316
Better/same/worse	6/1/0	6/1/0	4/3/0	5/0/2	5/0/2	4/3/0
MicroTEDAClus	**0.000**	**0.524**	**0.000**	**0.436**	**0.126**	**0.124**
DenStream	**0.822**	**0.567**	0.340	**0.405**	**0.196**	0.274
CluStream	**0.174**	**0.599**	**0.194**	0.589	0.338	0.320
StreamKM	**0.164**	0.860	0.451	0.569	0.341	0.301
AutoCloud	**0.020**	**0.135**	**0.040**	**0.052**	**0.073**	**0.070**
CEDAS	**0.211**	**0.000**	**0.142**	**0.000**	**0.000**	**0.000**
TEDACloud	0.760	**0.518**	0.541	**0.366**	**0.117**	**0.091**

In the Overall ARI, MicroTEDAClus+ is better than MicroTEDAClus in five datasets and indistinguishable from it in Cassini, and in the Steady-State ARI it is better in all six (Table 4.25). Against the benchmark models, it reverses the position of MicroTEDAClus in the streams with concept drift. Its Overall ARI is better than that of every other model in StreamBenchmark(1) and StreamBenchmark(2), and than all but TEDACloud in RandomRBFGeneratorEvents, and no model is better than it at the penultimate batch of these streams. In the static shapes, it is better than MicroTEDAClus, DenStream and the TEDA-based models, but worse than CluStream and StreamKM in Cassini and DS3, on both metrics, and worse than CluStream in the Overall ARI of DS4.

In RandomRBFGeneratorEvents, DS3 and DS4, the comparison with MicroTEDAClus also carries a difference of search budget, since the reference implementation exhausts the time of the search before its trials in these datasets (Table 4.2). The validation of Section 4.5, in which every model completes its trials, shows that the gains over the *vectorized* model do not depend on that difference.

Table 4.26: Clustering quality of MicroTEDAClus+, MicroTEDAClus and the benchmark models in the scalability streams: medians over the runs of the main experiment.

Model	S1	S2	S4	S8	S16	S32	S64
Overall ARI							
MicroTEDAClus+	0.694	0.947	1.000	1.000	1.000	1.000	1.000
Better/same/worse	4/3/0	7/0/0	6/1/0	6/1/0	6/1/0	6/1/0	6/1/0
MicroTEDAClus	0.763	**0.861**	**0.860**	**0.867**	**0.883**	**0.865**	**0.843**
DenStream	**0.000**	**0.690**	**0.442**	**0.426**	**0.000**	**0.000**	**0.000**
CluStream	0.661	**0.887**	**0.895**	**0.913**	**0.953**	**0.980**	**0.991**
StreamKM	0.665	**0.887**	**0.927**	**0.953**	**0.971**	**0.983**	**0.989**
AutoCloud	**0.325**	**0.446**	**0.405**	**0.426**	**0.435**	**0.443**	**0.445**
CEDAS	**0.511**	**0.932**	1.000	1.000	1.000	1.000	1.000
TEDACloud	**0.495**	**0.536**	**0.565**	**0.924**	**0.567**	**0.503**	**0.586**
Steady-State ARI							
MicroTEDAClus+	0.677	0.950	1.000	1.000	1.000	1.000	1.000
Better/same/worse	4/3/0	7/0/0	7/0/0	6/1/0	6/1/0	6/1/0	6/1/0
MicroTEDAClus	0.727	**0.857**	**0.856**	**0.847**	**0.810**	**0.462**	**0.000**
DenStream	**0.000**	**0.768**	**0.493**	**0.454**	**0.000**	**0.000**	**0.000**
CluStream	0.677	**0.873**	**0.894**	**0.916**	**0.960**	**0.980**	**0.991**
StreamKM	0.621	**0.903**	**0.934**	**0.957**	**0.972**	**0.985**	**0.990**
AutoCloud	**0.280**	**0.419**	**0.372**	**0.379**	**0.421**	**0.419**	**0.433**
CEDAS	**0.508**	**0.885**	**1.000**	1.000	1.000	1.000	1.000
TEDACloud	**0.532**	**0.583**	**0.606**	**0.903**	**0.598**	**0.529**	**0.510**

In the scalability streams, MicroTEDAClus+ is better than every other model from two dimensions on, on both metrics, except CEDAS, which reaches the same median

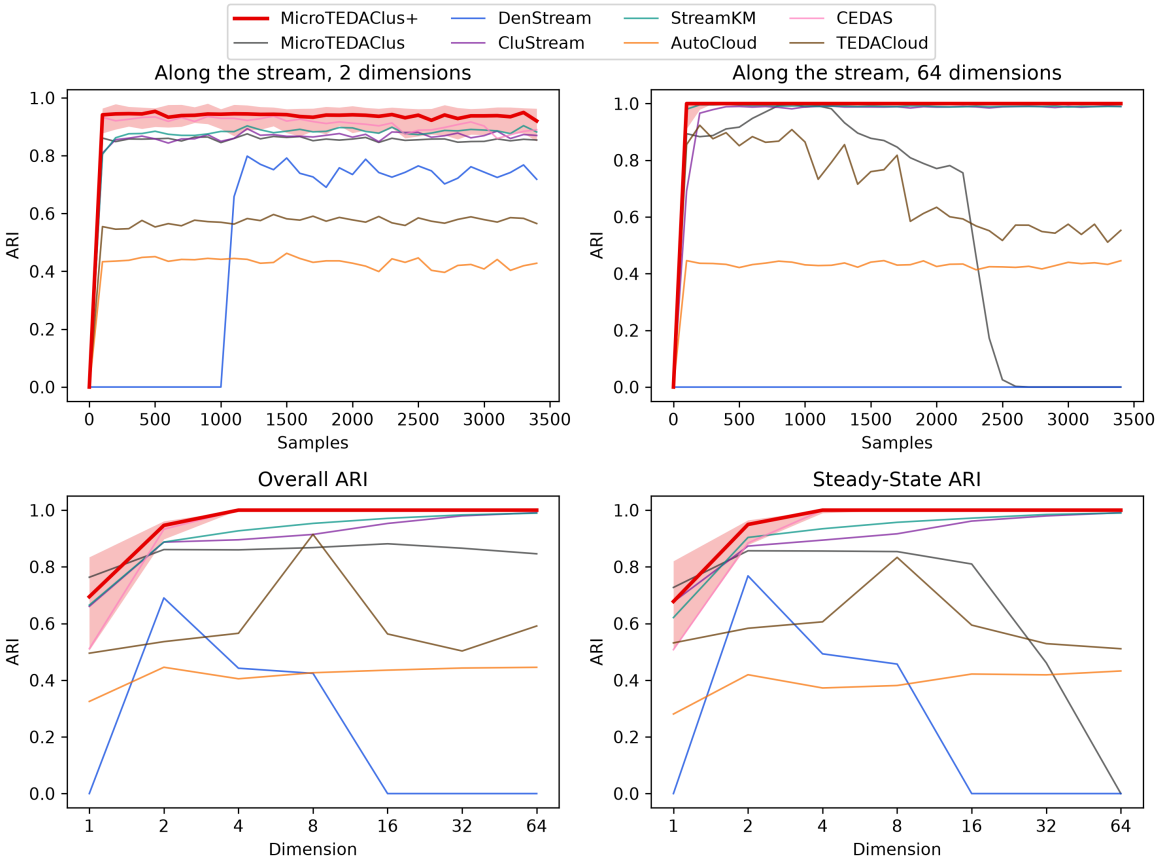


Figure 4.2: ARI of MicroTEDAClus+, MicroTEDAClus and the benchmark models in the scalability streams: along the stream with 2 and 64 dimensions, and the Overall and Steady-State ARI against the dimension. Medians, with the interquartile band of MicroTEDAClus+, over 94 runs.

of 1.000 and is indistinguishable from it from four or eight dimensions on (Table 4.26). Along the stream with 64 dimensions, the ARI of MicroTEDAClus collapses before the end, and that of MicroTEDAClus+ does not (Figure 4.2). The stream with one dimension is the exception: there, MicroTEDAClus+ is indistinguishable from MicroTEDAClus, CluStream and StreamKM, with a lower median than MicroTEDAClus. No one-dimensional stream took part in the development of the model.

4.8.2 Cost

MicroTEDAClus+ is faster than every other model in both stages of every dataset, except in the learning stage of StreamBenchmark(1) and RandomRBFGeneratorEvents, where it is indistinguishable from CEDAS (Table 4.27). Most of its prediction times print as zero because they are at or below the 10 ms resolution of the analysis. Its learning time is one to two orders of magnitude below that of MicroTEDAClus in four

Table 4.27: Execution time and memory footprint of MicroTEDAClus+, MicroTEDAClus and the benchmark models: medians over the runs of the main experiment.

Model	B1	B2	RBF	CSN	DS3	DS4
Learning time [ms]						
MicroTEDAClus+	30	10	70	10	30	30
Better/same/worse	6/1/0	7/0/0	6/1/0	7/0/0	7/0/0	7/0/0
MicroTEDAClus	**1000**	**570**	**8210**	**590**	**27850**	**18815**
DenStream	**180**	**200**	**200**	**190**	**210**	**210**
CluStream	**230**	**140**	**4260**	**120**	**380**	**375**
StreamKM	**60**	**30**	**120**	**25**	**120**	**105**
AutoCloud	**12910**	**50**	**460**	**16910**	**445**	**97150**
CEDAS	30	**20**	90	**30**	**90**	**70**
TEDACloud	**110**	**50**	**530**	**270**	**360**	**310**
Prediction time [ms]						
MicroTEDAClus+	0	0	10	0	0	0
Better/same/worse	7/0/0	7/0/0	7/0/0	7/0/0	7/0/0	7/0/0
MicroTEDAClus	**100**	**50**	**800**	**60**	**480**	**445**
DenStream	**90**	**140**	**150**	**135**	**70**	**110**
CluStream	**100**	**80**	**1220**	**80**	**260**	**200**
StreamKM	**10**	**10**	**30**	**10**	**30**	**30**
AutoCloud	**4770**	**20**	**160**	**6200**	**160**	**35680**
CEDAS	**20**	**10**	**60**	**20**	**70**	**50**
TEDACloud	**40**	**20**	**150**	**70**	**110**	**100**
Maximum memory footprint [KiB]						
MicroTEDAClus+	25	13	25	13	13	13
Better/same/worse	2/3/2	4/1/2	3/1/3	5/0/2	3/0/4	4/0/3
MicroTEDAClus	**277**	**141**	**1030**	**140**	**702**	**628**
DenStream	15	**28**	10	**24**	10	10
CluStream	**209**	**152**	**1066**	**148**	**218**	**201**
StreamKM	27	**28**	**84**	**31**	**35**	**38**
AutoCloud	654	3	11	**706**	9	**1941**
CEDAS	8	3	9	5	8	8
TEDACloud	6	3	11	12	7	7
Steady-state memory footprint [KiB]						
MicroTEDAClus+	25	13	24	13	13	13
Better/same/worse	2/1/4	4/1/2	3/1/3	5/0/2	3/0/4	4/0/3
MicroTEDAClus	**155**	**106**	**423**	**118**	**698**	**626**
DenStream	12	**23**	9	**18**	6	9
CluStream	**209**	**152**	**1066**	**148**	**218**	**201**
StreamKM	16	**17**	**31**	**17**	**20**	**28**
AutoCloud	629	2	9	**685**	6	**1914**
CEDAS	7	3	8	5	8	8
TEDACloud	6	2	9	8	3	4

datasets and close to three in the Chameleon datasets, which combines the optimization of Section 4.2 with the number of microclusters that each tuned model keeps, the driver of the cost (Section 4.5.2). As in Section 3.8, these comparisons measure implementations as much as algorithms: MicroTEDAClus+ runs the sequential part of the algorithm in compiled code, whose contribution Section 4.2.2 measures.

Its memory footprint is smaller than those of MicroTEDAClus and CluStream in every dataset and larger than those of CEDAS and TEDACloud in every dataset. The model stores the microclusters in arrays that always have room for one new microcluster per sample of the next batch and that are enlarged to twice the required size when that room runs out, so the footprint we measure is that of the allocated capacity, the same for every run that never keeps more than 100 microclusters.

4.8.3 Hyperparameters of MicroTEDAClus+

Table 4.28: Hyperparameters selected for MicroTEDAClus+: median [QCD] over the runs of the main experiment.

Model	Hyperparameter	B1	B2	RBF	CSN	DS3	DS4
MicroTEDAClus+	relative variance limit	0.00364 [0.98]	0.0348 [0.75]	0.0172 [0.74]	0.058 [0.41]	0.0622 [0.07]	0.0599 [0.19]
	decay	629 [0.78]	64.5 [0.50]	896 [0.85]	35 [0.80]	34 [0.30]	32 [0.79]
	noise distance	7.32 [0.16]	6.51 [0.27]	5.72 [0.20]	7.58 [0.18]	7.56 [0.16]	6.67 [0.19]

The relative variance limit selected for MicroTEDAClus+ spans about one order of magnitude across the datasets, against four for the absolute limit of MicroTEDAClus (Tables 3.5 and 4.28), and is nearly the same in the three static datasets, whose coordinates differ in scale by two orders of magnitude: the relative limit removes the dependence on the scale of the data. Its dispersion and the median decay are larger in the streams with concept drift, and the noise distance has a low dispersion in every dataset.

4.8.4 Summary

In the main experiment, MicroTEDAClus+ improves on MicroTEDAClus and changes its position among the benchmark models. It is better than MicroTEDAClus in the Overall ARI of five of the six datasets and in the Steady-State ARI of all six. No benchmark model is better than it in the streams with concept drift, where MicroTEDAClus was worse than most of them, and in the scalability streams it is better than every model but CEDAS from two dimensions on, without the collapse of MicroTEDAClus. It is the fastest model in both stages of every dataset, tied with CEDAS in the learning stage of two, and its memory footprint is smaller than those of MicroTEDAClus and

CluStream and larger than those of CEDAS and TEDACloud. Its variance limit no longer depends on the scale of the data. The weakness that remains is the clustering of static shapes, where CluStream and StreamKM remain better and which Section 4.6 relates to the configurations that the search selects.

Chapter 5

Conclusions

In this work, we set out to reassess MicroTEDAClus, an evolving clustering algorithm for data streams built on the TEDA framework, and to propose an improved model from that reassessment.

5.1 Final Remarks

The formulation of MicroTEDAClus has issues of its own, an order-dependent density, an outlier condition that cannot hold for microclusters with two samples and an unjustified scaling by the dimension (Section 2.2.1), and the implementation that produced the published results departs from it (Section 3.1). The methodology of the original experiments leaks information about the streams, makes most batches trivial to cluster and evaluates each batch after learning it, and its benchmark models ran through layers of interface that inflated their times (Chapter 3). Under a methodology that removes these conditions for every model alike, the original model is not on par with the benchmark models of the original work, and it does not adapt better than them to concept drift: at the penultimate batch it is better than none of the established benchmark models in any dataset, and in the scalability streams of high dimension its quality collapses, which we trace to the dimensionality penalty (Section 3.8).

MicroTEDAClus+, which we defined by controlled experiments on separate development and validation runs, differs from MicroTEDAClus in the five points of Table 4.24. It is better than MicroTEDAClus in the Overall ARI of five of the six datasets and in the Steady-State ARI of all six, faster in both stages and smaller in memory; no benchmark model is better than it in the streams with concept drift, and it is the fastest model of the main experiment (Section 4.8.4). Its weakness is the static shapes, where CluStream and StreamKM remain better.

We thus met the objectives of Section 1.2, within the limits that follow.

5.2 Limits of the Evidence

Four limits qualify these results.

- **Streams that repeat.** The Chameleon datasets are fixed files (Section 4.1.3). Their runs repeat the search and not the stream, so their dispersion is that of the search, and their validation confirms the outcome of the search more than that of the model.
- **The search as a source of variation.** Every result measures a model as its protocol tunes it. Two searches of a single algorithm give medians that differ without significance (Table 4.18): differences of that size are not effects of a model, and only the tests over many runs separate the effects from this variation.
- **Cost.** The execution time and the memory measure implementations and not only algorithms, as Section 4.8.2 notes and as the reimplementation of Section 2.4 shows for a single algorithm, and the models of a comparison are implemented in different libraries.
- **Datasets.** The datasets are streams and shapes of two dimensions and scalability streams of up to sixty-four, all synthetic. Our conclusions extend to data of that kind and not beyond.

5.3 Future Work

- **Hyperparameter selection.** The search selects configurations worse than those its space holds (Section 4.6), and its outcome varies with its seed (Table 4.18). An objective that observes a longer segment of the stream, more trials or repeated searches could reduce these effects, applied to every model of a comparison alike.
- **The static shapes.** They are where the model stays behind CluStream and StreamKM (Table 4.25), and where the probe of Section 4.6 finds far better configurations than the search selects; they are the main case to understand.
- **The noise label.** It improved the model on streams that carry noise (Section 4.4). Searching whether to label noise, as one more hyperparameter, would let a single model serve data whose classes cover every sample.
- **Explore its strengths.** It predicts faster than every benchmark model of the main experiment (Table 4.27), which suits applications in which labels must be returned quickly.

References

[1] AGGARWAL, C. C. et al. A framework for clustering evolving data streams. In: VLDB ENDOWMENT. *Proceedings of the 29th International Conference on Very Large Data Bases - Volume 29*. [S.l.], 2003. p. 81–92.

[2] CAO, F. et al. Density-based clustering over an evolving data stream with noise. In: SIAM. *Proceedings of the 2006 SIAM International Conference on Data Mining*. [S.l.], 2006. p. 328–339.

[3] WANG, X. et al. Data stream clustering: An in-depth empirical study. *Proc. ACM Manag. Data*, Association for Computing Machinery, New York, NY, USA, v. 1, n. 2, jun. 2023. Disponível em: <https://doi.org/10.1145/3589307>.

[4] MAIA, J. et al. Evolving clustering algorithm based on mixture of typicalities for stream data mining. *Future Generation Computer Systems*, v. 106, p. 672–684, 2020. ISSN 0167-739X. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0167739X19312786>.

[5] ANGELOV, P. Outside the box: an alternative data analytics framework. *Journal of Automation, Mobile Robotics and Intelligent Systems*, p. 29–35, 2014.

[6] GALINDO, J. C. F. *Mineração de Dados Aplicada a Problemas de Last-Mile na Logística*. Tese (Doutorado) — Escola de Engenharia, Universidade Federal de Minas Gerais, Belo Horizonte, MG, dez. 2021.

[7] CARDOSO, M. B. dos R. *Implementação do algoritmo MicroTEDAclus para classificação de dados*. Monografia (Graduação em Engenharia de Controle e Automação) — Escola de Engenharia, Universidade Federal de Minas Gerais, Belo Horizonte, MG, jul. 2022.

[8] KAUFMAN, S. et al. Leakage in data mining: Formulation, detection, and avoidance. *ACM Trans. Knowl. Discov. Data*, Association for Computing Machinery, New York, NY, USA, v. 6, n. 4, dez. 2012. ISSN 1556-4681. Disponível em: <https://doi.org/10.1145/2382577.2382579>.

[9] KAPOOR, S.; NARAYANAN, A. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, Elsevier, v. 4, n. 9, p. 100804, 2023. ISSN 2666-3899. Disponível em: <https://doi.org/10.1016/j.patter.2023.100804>.

[10] MONTIEL, J. et al. River: machine learning for streaming data in python. *Journal of Machine Learning Research*, v. 22, n. 110, p. 1–8, 2021. Disponível em: <https://jmlr.org/papers/v22/20-1380.html>.

[11] CARVALHO, E. H. B. de. *microtedaclus-reassessment*. 2026. https://github.com/eduardo-ufmg/microtedaclus-reassessment/tree/thesis-defense. Code of the experiments and sources of this thesis, at the tag thesis-defense.

[12] MAHALANOBIS, P. C. Reprint of: Mahalanobis, p.c. (1936) "on the generalised distance in statistics.". *Sankhya A*, v. 80, n. 1, p. 1–7, Dec 2018. ISSN 0976-8378. Disponível em: <https://doi.org/10.1007/s13171-019-00164-5>.

[13] ESTER, M. et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In: AAAI PRESS. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*. [S.l.], 1996. p. 226–231.

[14] ANGELOV, P. Anomaly detection based on eccentricity analysis. In: *2014 IEEE Symposium on Evolving and Autonomous Learning Systems (EALS)*. [S.l.: s.n.], 2014. p. 1–8.

[15] CARVALHO, E. H. B. de. *evolving_clustering*. 2026. https://github.com/eduardo-ufmg/evolving_clustering. Fork of Carlos Alberto Severiano's repository available at https://github.com/cseveriano/evolving_clustering. Accessed at commit 50dd2b4.

[16] ACKERMANN, M. R. et al. Streamkm++: A clustering algorithm for data streams. *ACM Journal of Experimental Algorithmics*, v. 17, p. 2.1–2.30, 2012.

[17] HYDE, R.; ANGELOV, P.; MACKENZIE, A. R. Fully online clustering of evolving data streams into arbitrarily shaped clusters. *Information Sciences*, Elsevier, v. 382–383, p. 96–114, 2017.

[18] AKINOSHO, T.; TABANE, E.; WANG, Z. A comparative analysis of data stream clustering algorithms. *International Journal of Computing*, p. 439–446, 12 2023.

[19] REZAEI, Z.; SAJEDI, H.; HASHEMI, M. Data stream clustering with concept drift using fractal dimension. *Knowledge and Information Systems*, v. 67, n. 10, p. 8715–8749, Oct 2025. ISSN 0219-3116. Disponível em: <https://doi.org/10.1007/s10115-025-02484-5>.

[20] MUTAMBIK, I. An entropy-based clustering algorithm for real-time high-dimensional iot data streams. *Sensors*, v. 24, n. 22, 2024. ISSN 1424-8220. Disponível em: <https://www.mdpi.com/1424-8220/24/22/7412>.

[21] JAHN, P. et al. Going offline: An evaluation of the offline phase in stream clustering. In: RIBEIRO, R. P. et al. (Ed.). *Machine Learning and Knowledge Discovery in Databases. Research Track*. Cham: Springer Nature Switzerland, 2026. p. 308–326. ISBN 978-3-032-06109-6.

[22] HAHSLER, M.; FORREST, J. *streamMOA: Interface for MOA Stream Clustering Algorithms*. [S.l.], 2024. R package version 1.3-1. Disponível em: <https://CRAN.R-project.org/package=streamMOA>.

[23] BEZERRA, C. G. et al. A new evolving clustering algorithm for online data streams. In: *2016 IEEE Conference on Evolving and Adaptive Intelligent Systems (EAIS)*. [S.l.: s.n.], 2016. p. 162–168.

[24] BEZERRA, C. G. et al. An evolving approach to data streams clustering based on typicality and eccentricity data analytics. *Information Sciences*, v. 518, p. 13–28, 2020. ISSN 0020-0255. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0020025519311363>.

[25] ZHANG, X.; SHEN, X.; OUYANG, T. Extension of dbscan in online clustering: An approach based on three-layer granular models. *Applied Sciences*, v. 12, n. 19, 2022. ISSN 2076-3417. Disponível em: <https://www.mdpi.com/2076-3417/12/19/9402>.

[26] NGUYEN-LE, H.-H. et al. Beyond binary classification: A semi-supervised approach to generalized ai-generated image detection. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. [S.l.: s.n.], 2026. v. 40, n. 42, p. 35733–35741.

[27] ZUBAROğLU, A.; ATALAY, V. Online embedding and clustering of evolving data streams. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, v. 16, n. 1, p. 29–44, 2023. First published online: 6 July 2022.

[28] KARYPIS, G.; HAN, E.-H.; KUMAR, V. Chameleon: Hierarchical clustering using dynamic modeling. *Computer*, IEEE, v. 32, n. 8, p. 68–75, 1999.

[29] DAWID, A. P. Present position and potential developments: Some personal views: Statistical theory: The prequential approach. *Journal of the Royal Statistical Society. Series A (General)*, Wiley, v. 147, n. 2, p. 278–292, 1984.

[30] ARIAS-GARCIA, J. et al. Improved design for hardware implementation of graph-based large margin classifiers for embedded edge computing. *IEEE Transactions on Neural Networks and Learning Systems*, v. 35, n. 1, p. 1320–1329, 2024.

[31] HALKIDI, M.; BATISTAKIS, Y.; VAZIRGIANNIS, M. On clustering validation techniques. *Journal of Intelligent Information Systems*, Kluwer Academic Publishers, v. 17, n. 2–3, p. 107–145, 2001. ISSN 0925-9902.

[32] STONE, M. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B*, v. 36, n. 2, p. 111–133, 1974.

[33] KOHAVI, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*. [S.l.: s.n.], 1995. v. 2, p. 1137–1143.

[34] HUBERT, L.; ARABIE, P. Comparing partitions. *Journal of Classification*, v. 2, n. 1, p. 193–218, 1985.

[35] OZAKI, Y.; WATANABE, S.; YANASE, T. OptunaHub: A platform for black-box optimization. *arXiv preprint arXiv:2510.02798*, 2025.

[36] SATOPAA, V. et al. Finding a "kneedle" in a haystack: Detecting knee points in system behavior. In: *2011 31st International Conference on Distributed Computing Systems Workshops*. [S.l.: s.n.], 2011. p. 166–171.

[37] DEMŠAR, J. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine learning research*, v. 7, n. Jan, p. 1–30, 2006.

[38] WILCOXON, F. Individual comparisons by ranking methods. *Biometrics bulletin*, JSTOR, v. 1, n. 6, p. 80–83, 1945.

[39] HOLM, S. A simple sequentially rejective multiple test procedure. *Scandinavian journal of statistics*, JSTOR, v. 6, n. 2, p. 65–70, 1979.

[40] CLIFF, N. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological bulletin*, American Psychological Association, v. 114, n. 3, p. 494–509, 1993.

[41] ROMANO, J. et al. Appropriate statistics for exploring effect size and power in research. In: *Annual meeting of the Florida Educational Research Association*. [S.l.: s.n.], 2006.

[42] ROSENBERG, A.; HIRSCHBERG, J. V-measure: A conditional entropy-based external cluster evaluation measure. In: *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*. Prague, Czech Republic: Association for Computational Linguistics, 2007. p. 410–420.